

BAR-ILAN UNIVERSITY
FACULTY OF ENGINEERING
FINAL PROJECT IN COMPUTER ENGINEERING
COURSE 83453 | PROJECT 323

FinWatch

An Extensible BLE Smartwatch Platform with a
Modular I²C Expansion Port

[TODO: Hebrew title]

Authors:

Omer Lazaros [TODO: ID]

Chen Leibov [TODO: ID]

Track: Hardware design

Academic supervisor: Prof. Leonid Yavitz leonid.yavitz@biu.ac.il

Project mentor: David Freud david.freud@mail.huji.ac.il

Date of submission: September 24, 2026 / [TODO: Hebrew date]

*Submitted as part of the undergraduate requirements
Faculty of Engineering, Bar-Ilan University*

הפקולטה להנדסה
ע"ש אלכסנדר קופקין
אוניברסיטת בר-אילן



Abstract

Wrist-worn wearables have become a general-purpose sensing platform, but nearly all of them, commercial and open-source alike, are sealed products: the set of sensors is fixed at manufacture, and adding a new measurement means designing a new watch.

This project asks whether a wearable can instead be built as an *extensible* platform, in which a user or a third party attaches new hardware to a finished, sealed device and the firmware discovers what it is at runtime.

We present **FinWatch**, a complete custom smartwatch built around the Nordic Semiconductor nRF5340 dual-core Bluetooth Low Energy SoC, with an nPM1300 power-management IC providing regulated 1.8 V and 3.3 V rails and Li-Po charging, an STMicroelectronics LSM6DSOX six-axis IMU, a 512 Mbit QSPI NOR flash, and an SPI display with capacitive touch.

The hardware was designed in KiCad as a four-layer PCB and the firmware was written for the Zephyr RTOS.

The distinguishing element is a four-pin pogo connector carrying 3.3 V, ground, and an I²C bus level-shifted from the SoC's 1.8 V domain, together with the **FinWatch Module Protocol (FWMP)**: a self-describing, CRC-protected descriptor format that lets any compliant I²C module announce its identity, capabilities, report sizes, and preferred polling rate.

The host treats every byte from an attached module as untrusted, validating magic numbers, protocol version, declared lengths, TLV bounds, and a CRC-16 before any field is used.

The firmware comprises roughly 2550 lines of C across nine subsystems and was verified functionally on an nRF5340-DK, including hot-plug enumeration, input/output report exchange, and six injected-fault cases that exercise every host-side rejection path.

The first PCB revision was manufactured and its power rails verified, but a missing SWD debug header prevented on-board firmware loading and forced a second revision; the design review that produced that revision is reported as a result in its own right.

The outcome is a working wearable platform and a documented expansion contract that a third party can implement without access to the watch firmware.

Acknowledgments

We would like to thank our project mentor, David Freud, for his guidance throughout the design, review, and bring-up of this project, and our academic supervisor, Prof. Leonid Yavitz, for his support and supervision. We also thank the Faculty of Engineering at Bar-Ilan University for providing the laboratory facilities and equipment used in this work, and the maintainers of the open tools this project depends on- KiCad, the Zephyr Project, and the nRF Connect SDK- whose excellent online courses, documentation and reference designs made a project of this scope feasible within a single academic year.

Contents

Abstract	1
Acknowledgments	2
1 Introduction	5
1.1 Background	5
1.2 Literature and market overview	6
1.3 Objective and contributions	7
1.4 Overview of this report	8
2 Background	8
2.1 Design fundamentals	8
2.1.1 Ground plane	8
2.1.2 Decoupling capacitors	9
2.1.3 Voltage step-down circuits	9
2.2 nRF5340	11
2.2.1 Network core	11
2.2.2 Application core	11
2.3 Communication protocols	12
2.3.1 UART	12
2.3.2 SPI	13
2.3.3 I ² C	14
2.3.4 BLE	14
2.4 Zephyr real-time operating system (RTOS)	15
2.5 Inertial measurement and MEMS: the BMI270	17
2.6 Power management IC: the nPM1300	18
3 System design overview	21
3.1 Requirements	21
3.2 System architecture	22
3.3 Component selection	23
3.4 Power budget	24
3.5 Firmware architecture	24
3.6 Use cases	25
4 The FinWatch Module Protocol (FWMP)	26
4.1 The electrical contract	26
4.2 The logical contract: register map	27
4.3 The descriptor: self-description through TLVs	28
4.4 Enumeration and the trust boundary	28
4.5 The behavioural contract: state machine	29
4.6 Adaptive polling policy	31
4.7 Writing a compliant module	31
4.8 Summary	32

5	Implementation	33
5.1	Schematic capture	33
5.2	PCB layout	34
5.2.1	Via-in-pad and the cost of fine-pitch escape routing	35
5.3	Mechanical design	37
5.4	Firmware	37
6	Design challenges and revisions	40
6.1	Revision A defect: no debug access	40
6.2	Component availability	40
6.3	The two-voltage-domain problem	41
6.4	No interrupt line on the expansion port	41
6.5	Hot-plug on an open-drain bus	42
6.6	Storage subsystem partition-ID collision	42
6.7	Pre-fabrication design-review checklist	42
7	Verification and results	43
7.1	Experiment 1 - Functional verification on the nRF5340-DK	43
7.2	Experiment 2 - Expansion protocol, nominal path	43
7.3	Experiment 3 - Expansion protocol, fault injection	44
7.4	Experiment 4 - Revision A electrical bring-up	44
7.5	Experiment 5 - Battery life estimate	45
7.6	Experiment 6 - Cost of interrupt-driven wake	45
7.7	Threats to validity	46
8	Discussion and comparison with prior work	47
8.1	Comparison with existing wearable platforms	47
8.2	ZSWatch in particular	47
8.3	Comparison with existing expansion ecosystems	48
8.4	What is new	49
8.5	Limitations	49
9	Future research directions	50
10	Conclusion	51
	References	52
A	Appendix A - Firmware	53
B	Appendix B - Hardware documentation	54
C	Appendix C - Reference expansion module	55

1 Introduction

1.1 Background

A modern smartwatch is, at its core, a microcontroller with a radio, a battery, a display, and a small collection of sensors, packaged so that it can be worn continuously.

Over the past decade this form factor has moved well beyond telling the time: it is now used for continuous health monitoring, fitness tracking, industrial telemetry, and as a general-purpose personal sensing platform.

What makes the category interesting from an engineering standpoint is the severity of its constraints.

The device must fit on a wrist, so board area and battery capacity are fixed and small; it must run for days on that battery, so average current consumption is measured in microamperes rather than milliamperes; it must maintain a wireless link, so a sensitive 2.4 GHz radio has to coexist on the same substrate as switching regulators and a display backlight; and it must remain responsive to the user, so the firmware has to service a display, a touch panel, a sensor, and a radio stack concurrently.

Each of these constraints pulls against the others. Efficient power conversion requires switching regulators, which generate the electromagnetic interference that degrades radio sensitivity.

Long battery life requires the processor to sleep as much as possible, which conflicts with a display that must redraw and a sensor that must be sampled.

Small board area forces components closer together, which worsens both thermal behaviour and crosstalk.

A wearable design is therefore less an exercise in selecting parts than in negotiating between mutually hostile requirements, and this negotiation is the substance of the present project.

There is a second, less technical constraint that motivates this work specifically.

A wearable is a sealed object. Once it is assembled, glued, and made water-resistant, its sensor complement is frozen for the lifetime of the product.

If a user, a researcher, or a workplace needs a measurement the manufacturer did not anticipate—an environmental gas sensor, an electrochemical probe, an RFID reader, a specialised medical transducer, the only available answer is a different watch.

The cost of that answer is not merely commercial: it means that every new wrist-worn measurement idea must re-solve power management, radio, display, mechanical design, and certification from scratch before it can be tested at all.

1.2 Literature and market overview

Approaches to building wrist-worn devices fall into three broad families.

Closed commercial products. Devices such as those from the major consumer vendors are highly optimised for battery life, size, and manufacturing cost. Their internal buses are inaccessible, their firmware is proprietary, and their sensor set is fixed. They are excellent products and useless as platforms.

Open-source wearables. The PineTime and Bangle.js represent the open end of the spectrum: both publish schematics and allow user firmware, and both have active communities. Architecturally they are close to this project- an nRF52-class SoC, an SPI display, an accelerometer, a Li-Po cell- and their existence is what makes the hardware side of this project tractable as a student effort.

But they are open in the sense of *modifiable firmware*, not *extensible hardware*. Neither exposes a documented electrical and logical interface through which a sealed device can acquire new sensing capability after manufacture.

Adding a sensor to a PineTime means opening it and soldering to test pads, at which point water resistance and mechanical integrity are gone.

ZSWatch. The closest prior work to this project is ZSWatch, an open-source smartwatch built on an nRF5340 (as a u-blox NORA-B10 module) running Zephyr, with an nPM1300 PMIC, a Bosch BMI270 IMU, a Macronix MX25U51245G external flash, and a 240×240 round touch display.

The convergence with the present design is close enough to be worth stating plainly: the same SoC, the same RTOS, the same PMIC, the same external flash part, and- as originally specified- the same IMU. This is less coincidence than evidence that the component space for an open Zephyr wearable is narrow, and that two independent projects arrived at the same defensible answers within it. ZSWatch is considerably more mature than this project and carries a richer sensor complement, including barometric, magnetometer, ambient-light and microphone channels that this design does not have.

Where the two diverge is expansion. ZSWatch does offer an Extension PCB, but it plugs into the breakout headers of the separate *WatchDK* development board rather than into the finished watch, and it is a specific first-party add-on- heart rate and an audio codec- supported by drivers written for it. That is expansion in the development-kit sense, the same sense in which the nRF5340-DK is expandable, and it does not address a sealed device acquiring capability from hardware its firmware has never seen. ZSWatch's dock is a USB-C breakout for debugger and UART access rather than a peripheral bus, and no self-describing descriptor or runtime discovery mechanism is documented for either interface.

Vendor development kits. The nRF5340-DK and the Nordic Thingy:53 sit at the opposite extreme: every pin is broken out, every bus is probeable, and expansion is trivial through Arduino-style or pin headers.

They are deliberately not wearable- they are large, run from a coin cell or USB, and have no enclosure- so they answer the extensibility question by abandoning the form factor that created it.

The gap. Modular *phone* concepts (Project Ara and its descendants) attacked the same problem at a larger scale and largely failed on mechanical and connector complexity: a full high-speed modular bus is expensive in area, contacts, and power.

What has not been well explored is the modest version of that idea at wearable scale- a single low-speed bus, a handful of contacts, and a protocol thin enough that a hobbyist can implement it on an 8-bit microcontroller.

Standardised self-describing device protocols exist in adjacent domains (USB HID descriptors, I²C-based Qwiic/STEMMA QT ecosystems, and the SMBus device-identification registers), but the former is far too heavy for a wearable's power budget and the latter two standardise the *connector* without standardising *discovery*: a Qwiic module tells you nothing about itself, and the host must already contain a driver for it.

1.3 Objective and contributions

The objective of this project is to design, build, and verify a wearable computing platform that keeps the size, power, and integration constraints of a real smartwatch while exposing a documented, safe, and self-describing expansion interface, so that new hardware capability can be added to a finished device without modifying its firmware.

The concrete contributions are:

1. **FinWatch hardware.** A four-layer custom PCB integrating an nRF5340 SoC with its RF matching network and dual crystals, an nPM1300 PMIC providing two switched-mode rails plus Li-Po charging and fuel gauging, a LSM6DSOX IMU, 512 Mbit of QSPI flash, an SPI LCD with capacitive touch, and two pogo-pin external interfaces. Designed in KiCad and manufactured (Section 5).
2. **The FinWatch Module Protocol (FWMP v1).** An I²C-based expansion contract comprising an electrical specification (four contacts: SDA, SCL, 3.3 V, GND), a register map, a CRC-protected TLV descriptor through which a module declares its identity, capabilities, report geometry, and preferred poll rate, and a host-side enumeration state machine that treats all module-supplied data as untrusted (Section 4).
3. **A Zephyr RTOS firmware** of roughly 2 550 lines of C across nine subsystems, implementing the watch application and the host side of FWMP, including an adaptive polling policy that trades latency against current consumption according to whether the user is looking at the module's screen (Section 5).
4. **A mock transport backend** that serves a canned descriptor and injects six classes of malformed descriptor, making every host-side rejection path reachable in testing without physical module hardware (Section 7).

5. **A documented design-review process**, including the two hardware defects that this project's first manufactured revision exhibited, the root-cause analysis of each, and the pre-fabrication checklist derived from them (Section 6).

1.4 Overview of this report

Section 2 presents the electronic and software foundations the rest of the report assumes: PCB design fundamentals, voltage conversion, the serial protocols used on the board, the nRF5340 architecture, MEMS inertial sensing, the nPM1300 PMIC, and the Zephyr RTOS.

Section 3 gives the high-level design: requirements, system architecture, component selection and its trade-offs, the power budget, the firmware thread model, and the principal use cases.

Section 4 specifies the expansion protocol in full- the contribution at the center of this project.

Section 5 describes the implementation: schematic capture, PCB layout, enclosure, firmware structure, and toolchain.

Section 6 reports the design problems encountered and how they were resolved, including the revision-A defects.

Section 7 presents the verification results.

Section 8 compares the outcome with prior work,

Section 9 discusses future directions, and

Section 10 concludes Firmware listings, hardware exports, and a reference module design appear in the appendices.

2 Background

This section collects the foundations used throughout the rest of the report: the printed-circuit-board design principles that constrain the layout, the voltage-conversion topologies that determine the power architecture, the serial protocols that connect the components, and the three major integrated circuits and one operating system on which the design rests. Later sections assume this material and do not repeat it.

2.1 Design fundamentals

2.1.1 Ground plane

Current follows the path of least impedance. To ensure return currents have a direct, low-impedance path back to the source, a dedicated layer of metal (the ground plane) is used.

By connecting all ground points to this plane, we minimise the loop area between signals and their returns, significantly reducing noise and electromagnetic interference (EMI).

In a design that places a switching regulator and a 2.4 GHz radio on the same substrate, this is not a refinement but a precondition: the loop area of the buck converter's switching node is the dominant radiator on the board, and the antenna is a few centimetres away from it.

2.1.2 Decoupling capacitors

Integrated circuits (ICs) often exhibit rapid spikes in current consumption during logic switching. Due to the inherent inductance and impedance of PCB traces, the power supply cannot always respond fast enough, which can cause the local voltage to drop below operational levels (brownout).

To mitigate this, decoupling capacitors are placed very close to the IC's power pins.

These capacitors act as local energy reservoirs, discharging stored charge to meet the immediate current demand and maintaining a stable voltage level for the device.

The nRF5340 in this design carries a bank of eleven 100 nF capacitors (C14–C24) distributed across its supply pins for exactly this reason.

2.1.3 Voltage step-down circuits

Low-dropout regulator (LDO). A low-dropout (LDO) regulator is a type of DC linear voltage regulator that can step down an input voltage to a stable output voltage, even when the input is very close to the desired output. It operates using an error amplifier (op-amp) that continuously adjusts the resistance of a pass transistor (usually a MOSFET) based on feedback from a voltage divider.

The output voltage is configured by the ratio of the feedback resistors:

$$V_{out} = \left(1 + \frac{R_1}{R_2}\right) V_{ref}$$

While LDOs are prized for their simplicity, low cost, and ability to provide a completely noise-free output, their primary disadvantage is poor efficiency when the difference between the input and output voltage is large.

Because an LDO acts as a variable resistor to drop the voltage, it dissipates all excess power as heat.

The power lost to heat is defined as:

$$P_{loss} = (V_{in} - V_{out}) \times I_{load}$$

Considering that the typical efficiency of an LDO is characterised by

$$\text{Efficiency} \approx \frac{V_{out}}{V_{in}}$$

dropping the voltage from 5 V to the 1.8 V required by the [nRF5340](#) results in an efficiency of

$$\frac{1.8}{5} = 36\%$$

that is, nearly two-thirds of the energy drawn from the battery would be converted directly into heat inside a sealed plastic case worn against skin. This calculation is the reason the design uses switching conversion throughout.

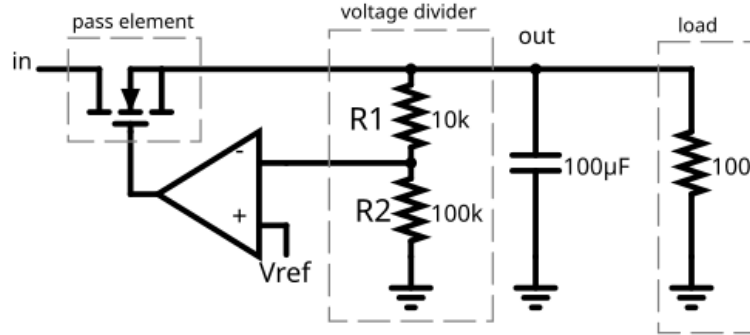


Figure 1: LDO circuit. The error amplifier drives the pass element so that the divided output matches V_{ref} ; the pass element dissipates $(V_{in} - V_{out})I_{load}$ as heat.

Buck converter (step-down switching regulator): To overcome the thermal inefficiencies of linear regulators, a buck converter is used.

A buck converter is a type of switched-mode power supply (SMPS) that efficiently steps down voltage using a completely different mechanism: instead of dissipating excess voltage as heat, it rapidly switches the input voltage on and off.

A standard buck converter circuit consists of a switching transistor, a diode (or a second transistor for synchronous bucking), an inductor, and a capacitor.

The transistor chops the input DC voltage into a high-frequency square wave. The inductor stores energy in a magnetic field during the “on” phase and releases it during the “off” phase, while the output capacitor smooths this chopped energy back into a steady DC voltage.

The efficiency of a buck converter frequently exceeds 90%, dramatically extending battery life in compact, power-constrained designs. However, this efficiency comes with distinct engineering trade-offs:

- **Complexity and size:** they require external inductors and larger capacitors, which consume more physical PCB area.
- **Switching noise:** the high-frequency switching (often between 1 MHz and 3 MHz) introduces voltage ripple and electromagnetic interference into the system. This noise can severely degrade sensitive analog circuits or RF communication modules if the PCB layout is not carefully routed.

The governing relationship for the ideal continuous-conduction case is

$$V_{out} = D \cdot V_{in}, \quad D = \frac{T_{on}}{T_{total}}$$

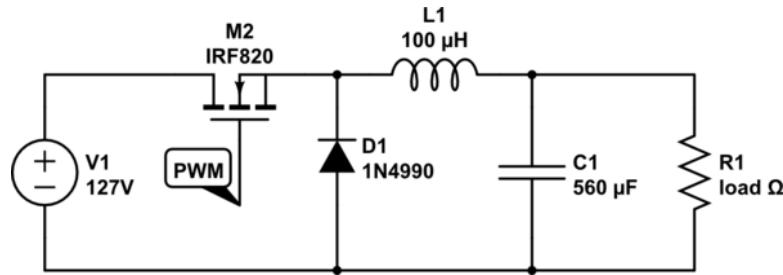


Figure 2: Buck converter circuit. Energy is transferred to the output through the inductor rather than dissipated across a pass element, which is what allows efficiencies above 90%.

2.2 nRF5340

The nRF5340 is a wireless ultra-low-power multicore SoC with two fully programmable Arm Cortex-M33 processors: a network core and an application core.

2.2.1 Network core

An Arm Cortex-M33 processor with a reduced feature set designed for ultra-low-power operation. It is used for radio communication and real-time processing tasks involving low-level radio protocol layers. Build target:

```
<board_target>/cpunet
```

2.2.2 Application core

A full-featured Arm Cortex-M33 processor that includes DSP instructions and an FPU. This core is used for tasks that require high performance and for application-level logic.

The M33 TrustZone, one of the Cortex-M Security Extensions (CMSE), can divide the application MCU into a Secure Processing Environment (SPE) and a Non-Secure Processing Environment (NSPE).

When the MCU boots, it always starts executing from the secure area.

Build targets:

```
<board_target>/cpuapp (TF-M disabled).
<board_target>/cpuapp/ns (TF-M enabled, SPE firmware alongside the NSPE firmware).
```

The split-core architecture matters for this project beyond nomenclature: it means the timing-critical BLE link layer runs on a core that the application firmware cannot stall.

A long-running expansion-port transaction on the application core- for example, a module holding SCL low- therefore cannot break an active Bluetooth connection.

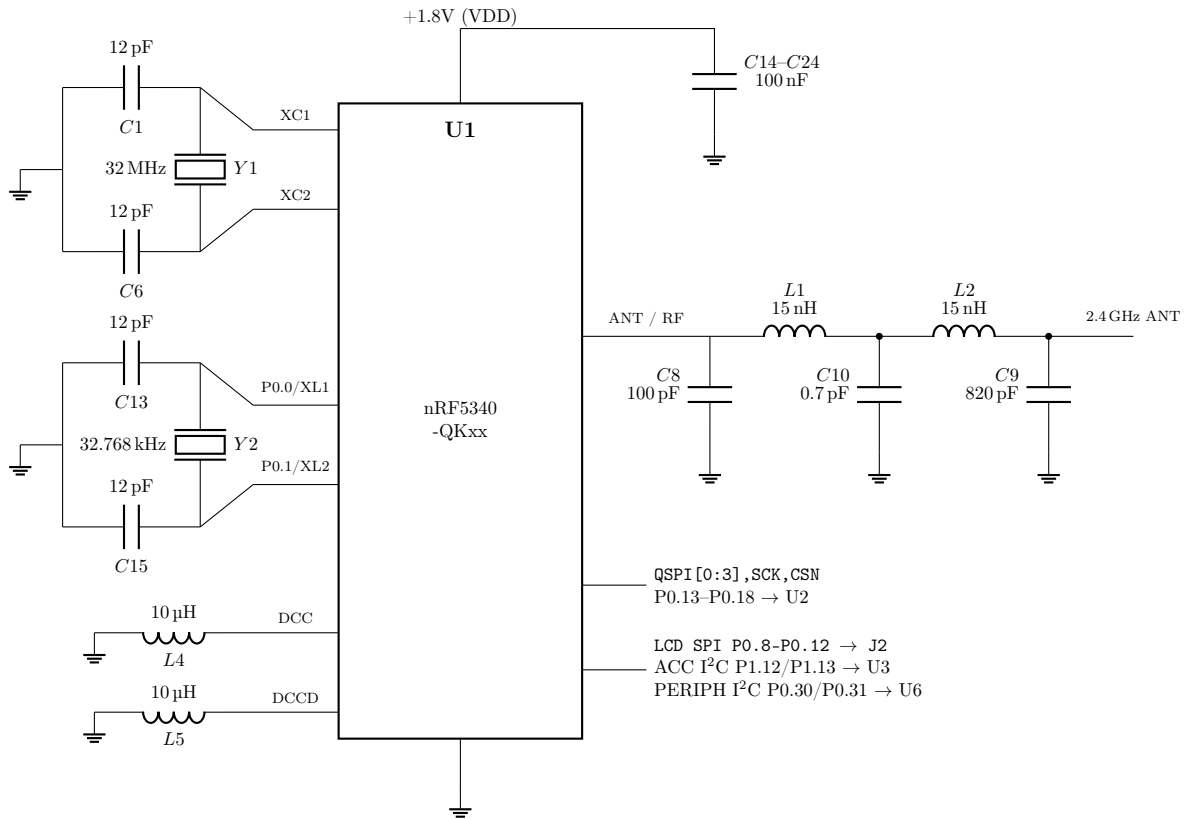


Figure 3: nRF5340 application SoC (U1). The 32 MHz (Y1) and 32.768 kHz (Y2) crystals use 12 pF loading capacitors; L4/L5 are the DCC/DCCD DC-DC converter inductors; L1/L2 with C8–C10 form the RF match to the on-board antenna. C14–C24 (100 nF) are the supply-pin decoupling bank. Digital pins are shown as their KiCad net labels.

2.3 Communication protocols

2.3.1 UART

UART, or **universal asynchronous receiver-transmitter**, is a serial communication protocol used widely between integrated circuits due to its simplicity, configurable data format, and transmission speeds.

UART works by sending data bits one by one, from the least to the most significant, framed by start and stop bits, so that precise timing is handled by the communication channel rather than a shared clock.

For successful UART communication, the following settings must match on both sides:

1. Voltage level
2. Baud rate (the rate at which bits are sent)
3. Parity bit
4. Data-bit size
5. Stop-bit size

6. Flow control

A UART frame consists of five elements:

- Idle (logical 1)
- Start bit (logical 0)
- Data bits - five to nine bits representing the payload
- Parity bit - an optional bit after the data bits, used for a parity check
- Stop (logical 1) - one or two bits representing the stop signal

In this project UART is used only as the firmware logging and console channel during development; no on-board peripheral communicates over it.

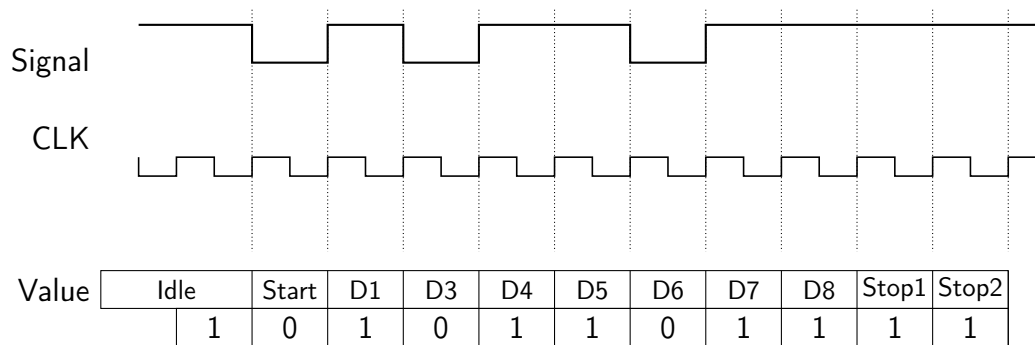


Figure 4: Example of a UART frame with one start bit, seven data bits, and two stop bits.

2.3.2 SPI

SPI (Serial Peripheral Interface) is a synchronous, full-duplex, four-wire serial communication interface used for short-distance communication between a controller and multiple target devices.

Unlike [I²C](#), which is limited by pull-up resistor speeds, SPI is limited only by the maximum clock frequency of the devices, making it the preferred choice for high-bandwidth applications like SD cards, LCDs, and high-speed sensors.

SPI uses four dedicated signal lines:

- SCLK (serial clock): the clock signal generated by the controller to synchronise data.
- MOSI (main out, sub in): the line used to send data from the controller to the target.
- MISO (main in, sub out): the line used to send data from the target back to the controller.
- CS/SS (chip select / slave select): a dedicated line for each target device. The controller pulls this line low to select the specific device it wants to talk to.

In this design SPI carries the display frame data, and its four-bit variant (QSPI) carries traffic to the external flash.

2.3.3 I²C

I²C is a synchronous, multi-controller / multi-target serial communication bus developed by Philips Semiconductors.

It is the industry standard for short-distance, intra-board communication between processors and low-speed peripherals.

I²C offers two primary advantages over similar protocols such as SPI:

- **Minimal pin count:** it requires only two signals, regardless of how many devices are on the bus.
- **Software addressing:** unlike SPI, which requires a dedicated select line for each peripheral, I²C uses unique 7-bit addresses to communicate with multiple targets on the same bus.

The bus consists of two bidirectional lines: the **serial data (SDA)** and the **serial clock (SCL)**.

Because the devices use open-drain outputs, both lines must be pulled up to V_{CC} via resistors (typically 4.7 k Ω or 10 k Ω).

These two properties are precisely why I²C was chosen as the expansion transport in Section 4: two contacts serve an unlimited number of future module types, and addressing is a software matter rather than an additional physical contact per device.

Its open-drain nature also has a consequence that Section 6 returns to: a module unplugged mid-transaction can leave SDA held low and jam the bus for every other participant.

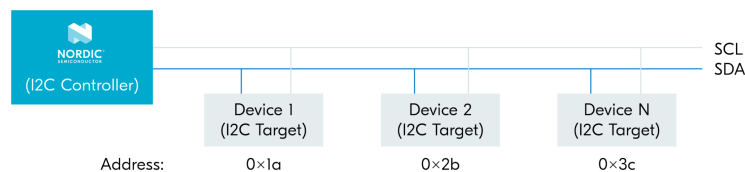


Figure 5: I²C bus topology. Open-drain devices pull the lines low; pull-up resistors restore the idle high level, and their value bounds the achievable bus speed.

2.3.4 BLE

Bluetooth Low Energy is a wireless personal-area-network technology designed to provide significantly reduced power consumption compared to “Classic” Bluetooth.

While Classic Bluetooth is intended for continuous data streaming, BLE is optimised for devices that transmit small amounts of data at infrequent intervals.

Main characteristics:

- **Low duty cycle:** BLE devices remain in a sleep state most of the time, waking only for short bursts to advertise their presence or exchange data, which makes the protocol very power efficient.
- **GATT architecture:** data is organised into a Generic Attribute Profile (GATT). This hierarchy consists of services (logical groupings of data) and characteristics (the actual data values, such as a sensor reading), allowing standardised communication across different manufacturers.
- **Connection roles:** the protocol distinguishes between the peripheral (the low-power sensor node) and the central (the powerful device, such as a smartphone, that manages the connection and processes data).

2.4 Zephyr real-time operating system (RTOS)

Historically, embedded firmware was written as “bare-metal” code, relying on a single infinite `while(1)` loop (a superloop) to sequentially poll sensors, manage timers, and handle communication.

While sufficient for simple microcontrollers, the superloop architecture breaks down in modern IoT and wearable devices that must concurrently manage a BLE radio stack, graphics rendering, sensor fusion, and strict power management.

To orchestrate this complexity, a **real-time operating system (RTOS)** is required. Zephyr is an open-source, highly scalable RTOS hosted by the Linux Foundation, and serves as the core foundation for advanced vendor toolchains, particularly for Arm Cortex-M architectures.

Pre-emptive multithreading and scheduling. At the heart of Zephyr is its pre-emptive scheduler. Instead of a single loop, application logic is divided into isolated, independent **threads**, each assigned a strict priority level.

The scheduler guarantees that the highest-priority thread ready to run always controls the CPU.

If a low-priority thread (for example, updating a display) is executing and a high-priority event occurs (for example, an incoming BLE packet), the scheduler instantly pre-empts the low-priority thread, saves its execution state, and hands control to the high-priority task.

This guarantees deterministic, real-time response latency, which is critical for maintaining wireless communication links.

Hardware abstraction: Devicetree and Kconfig. One of Zephyr’s most powerful paradigms is the strict separation of application logic from hardware description, borrowing heavily from the Linux kernel architecture:

- **Devicetree (.dts):** a hierarchical data structure that describes the physical PCB hardware. Instead of hardcoding GPIO pins, I²C addresses, or SPI frequencies in C code, developers define them in the Devicetree.
- **Kconfig:** a configuration system used to enable or disable specific software features, subsystems, and drivers at compile time, ensuring the final binary footprint remains as small as possible.

This architecture makes Zephyr firmware highly portable.

Transitioning code from a vendor evaluation board to a custom-designed PCB largely amounts to swapping the Devicetree overlay, leaving the core application logic untouched- a property this project depended on directly, since the firmware was developed and verified on an nRF5340-DK before the custom board existed.

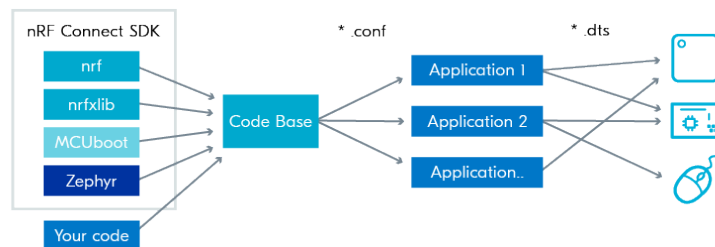


Figure 6: Zephyr build process. Devicetree describes the hardware and Kconfig selects the software features; together they produce a board-specific binary from board-independent application code.

Advanced power management. For battery-operated devices, maximising time between charges is the primary engineering constraint. Zephyr features a natively integrated power-management (PM) subsystem. It uses a **tickless kernel**, meaning the OS does not rely on a constant, battery-draining periodic timer to wake the CPU.

Instead, when all threads are idle or waiting for external events, Zephyr calculates the exact time until the next scheduled task and places the microcontroller into an ultra-low-power deep-sleep mode.

Furthermore, device PM allows the OS to selectively cut power to individual peripheral buses (such as shutting down an I²C sensor or SPI flash) when they are not actively in use.

Subsystem integration. Zephyr provides production-ready, well-tested subsystems out of the box, including file systems, USB device stacks, cryptographic libraries, and a fully qualified open-source BLE host stack. Zephyr’s architecture supports split-stack designs, making it well suited to dual-core SoCs where a dedicated network core handles the strict timing of the BLE radio while an application core manages the user interface and sensor data.

2.5 Inertial measurement and MEMS: the BMI270

To capture physical kinematics in three-dimensional space, modern embedded systems use an inertial measurement unit (IMU). The Bosch Sensortec BMI270 is a highly integrated, ultra-low-power system-in-package featuring a 3D digital accelerometer and a 3D digital gyroscope in a $2.5 \times 3.0 \times 0.8$ mm LGA-14 package, targeted at wearable and always-on applications. It is built using micro-electromechanical systems (MEMS) technology, in which microscopic mechanical structures are etched directly into the silicon alongside the analog front-end and digital logic.

Its headline specifications are given in Table 1. Two of them determine its place in this design: it operates directly from the board’s 1.8 V digital rail without a dedicated supply, and it speaks I^2C at a 7-bit address selected by the SDO pin, so it shares the existing sensor bus rather than requiring dedicated pins.

Parameter	LSM6DSOX
Package	LGA-14, $2.5 \times 3.0 \times 0.83$ mm
Supply (VDD)	1.71 V to 3.6 V
I/O supply (VDDIO)	1.62 V to 3.6 V
Digital interfaces	I^2C , SPI, MIPI I3C
I^2C address	0x6A / 0x6B (selected by SA0)
Accelerometer range	± 2 / ± 4 / ± 8 / ± 16 g
Gyroscope range	± 125 to ± 2000 °/s
Output data rate	up to 6.66 kHz
FIFO	9 KiB “smart” FIFO with timestamping
Embedded processing	Machine Learning Core, Finite State Machine, pedometer
Sensor hub	I^2C controller for up to 4 external sensors

Table 1: LSM6DSOX key parameters. The wide VDD range is what allows the part to sit directly on the 1.8 V digital rail alongside the SoC and the QSPI flash, with no additional regulator.

Working principle: accelerometer (linear motion). The accelerometer measures linear acceleration using capacitive sensing. Internally it contains a microscopic mechanical structure consisting of a tethered “proof mass” suspended by silicon springs between fixed electrodes. When the device experiences acceleration, the inertia of the proof mass causes it to lag behind the movement of the sensor housing. This displacement changes the distance (d) between the movable mass and the fixed electrodes, altering the capacitance according to the parallel-plate relation

$$C = \frac{\epsilon A}{d}$$

The analog front-end measures this minute change in capacitance (ΔC), converts it to a voltage, and passes it through an analog-to-digital converter to yield a digital acceleration value on the X, Y, or Z axis.

Working principle: gyroscope (angular velocity). The gyroscope measures the rate of rotation by exploiting the **Coriolis effect**. Inside the MEMS structure a proof mass is continuously driven to vibrate at a high resonant frequency in a specific plane. When the sensor rotates about an axis, the vibrating mass experiences the Coriolis force, which acts

perpendicular to both the direction of vibration and the axis of rotation. The Coriolis force is proportional to the angular velocity $\vec{\Omega}$:

$$\vec{F}_c = -2m(\vec{\Omega} \times \vec{v})$$

where m is the mass and $\vec{v}$ its instantaneous velocity. This force displaces the mass into a secondary sensing plane, which is again measured capacitively and digitised.

Advanced architecture: embedded processing and smart interrupts. While many IMUs simply output raw X, Y, and Z data over [I²C](#) or SPI, continuous polling of that data forces the main microcontroller ([nRF5340](#)) to remain awake, drawing significant battery power. The defining advantage of the LSM6DSOX is that it can do a substantial amount of that work itself, in three layers of increasing sophistication:

- **Fixed-function embedded features.** A hardware pedometer (step detector and step counter), tilt detection, significant-motion detection, free-fall, wake-up, and 6D/4D orientation are implemented in the sensor and require only register configuration. Each can be routed to one of the two interrupt pins, so the host is notified of an *event* rather than being handed a stream of samples to interpret.
- **Finite State Machine (FSM).** A set of programmable state machines that trigger on user-defined sequences of sensor conditions, for gesture recognition beyond the fixed-function set.
- **Machine Learning Core (MLC).** An in-sensor decision-tree classifier. Trees are trained offline on labelled motion data using ST's MEMS Studio tooling and loaded as a configuration (.ucf) file at initialisation; the sensor then performs continuous activity classification - walking, running, stationary - and raises an interrupt when the classified class changes.

The intended consequence for a wearable is that the IMU performs continuous, real-time context and gesture recognition at a small fraction of a milliampere, allowing the main MCU to remain in deep sleep until the sensor announces that a specific physical event has occurred. The 9 KiB FIFO reinforces this: even where the host must see raw samples, it can sleep while the sensor buffers them and wake once per batch rather than once per sample.

The firmware exploits the first of these three layers: the wake-up (activity) detector is configured to assert INT1 on motion, and the application core blocks until it fires rather than polling. Step counting itself still runs on the host, inside a bounded active window opened by that interrupt - the design is described in [Section 5](#), and moving it into the sensor's own pedometer is discussed in [Section 9](#).

2.6 Power management IC: the nPM1300

In ultra-low-power wearable and IoT designs, efficient power conversion and rigorous battery management are critical for maximising battery longevity while maintaining a compact form factor.

The Nordic Semiconductor nPM1300 is a dedicated, highly integrated power management integrated circuit (PMIC) designed to complement low-power microcontrollers such as the [nRF5340](#).

Rather than relying on scattered discrete regulators, battery chargers, and supervisor ICs, the nPM1300 consolidates the essential power-management building blocks into a single 5×5 mm QFN32 package.

It incorporates dual high-efficiency synchronous buck converters, two linear regulators / load switches, a full-featured linear Li-ion/Li-Po battery charger with automatic power-path management, an on-chip Coulomb counter for fuel gauging, and system supervisor logic.

Dual synchronous buck converters. Building on the switched-mode principles introduced in [Buck converter \(step-down switching regulator\)](#); the nPM1300 integrates two step-down switching regulators.

Unlike discrete buck implementations that require external power MOSFETs and control feedback loops, the nPM1300 embeds the switching transistors and the PWM/PFM control circuitry internally.

Each converter requires only a miniature external inductor ($L = 2.2 \mu\text{H}$) and filtering capacitor ($C = 10 \mu\text{F}$) to achieve conversion efficiencies exceeding 90% (Figure 7):

- **BUCK1 (1.8 V rail):** configured by external resistor $R_8 = 47 \text{ k}\Omega$ on the VSET1 pin. This rail powers the main digital logic, flash memory, and sensor interfaces, including the [nRF5340](#) core supply alongside the [Decoupling capacitors](#)-filtered peripheral rails.
- **BUCK2 (3.3 V rail):** configured by external resistor $R_9 = 250 \text{ k}\Omega$ on the VSET2 pin. This rail provides the higher operating voltage required by external peripherals, such as the display backlight, the off-board I/O headers, and - critically for this project - the expansion port (Figures 10 and 11).

During ultra-low-power sleep states the regulators automatically switch from PWM to PFM mode, reducing quiescent current to microampere levels.

Battery charging and power-path management. The nPM1300 incorporates an autonomous linear battery charger supporting single-cell lithium-ion and lithium-polymer chemistries connected via the battery terminal (J4):

- **CC/CV charging profile:** the charger operates in constant-current / constant-voltage mode, with programmable termination voltages (3.5 V to 4.45 V), configurable charge currents up to 800 mA, and automatic trickle charging for deeply depleted cells.
- **Dynamic power path:** the PMIC monitors the V_{BUS} input from the USB pogo-pin interface (Figure 11). When V_{BUS} (5 V) is present, power-path management routes power directly from the external source to the system buck regulators while directing the remaining current to charge the battery. When USB power is detached, the system transitions to battery operation with no supply glitch, preventing unexpected brownout resets on the [nRF5340](#).
- **Thermal safety (JEITA):** by interfacing with an external NTC thermistor, the charger continuously adjusts current limits and target voltages based on battery temperature, enforcing standard JEITA safety boundaries to prevent thermal runaway.

Fuel gauging and Coulomb counter. Accurate state-of-charge (SoC) estimation is notoriously difficult in lithium-based batteries because the open-circuit discharge voltage remains virtually flat throughout most of the discharge cycle.

Relying solely on voltage measurements leads to severe inaccuracies under dynamic load transitions. To overcome this limitation, the nPM1300 integrates an autonomous hardware **Coulomb counter**, which continuously samples the bidirectional current across an internal sense element and integrates the total charge over time:

$$Q(t) = Q_0 + \int_0^t I_{\text{batt}}(\tau) d\tau$$

Coupled with high-accuracy internal ADCs monitoring instantaneous battery voltage, load current, and die temperature, the fuel gauge provides reliable real-time capacity metrics to the host without requiring continuous, power-draining CPU sampling routines.

Host control, system supervisor, and Zephyr integration. The nPM1300 operates in close conjunction with the host processor:

- **Two-wire digital interface:** all internal registers, regulator set-points, charger telemetry, and interrupt masks are accessed by the [nRF5340](#) over the standard [I²C bus.](#)
- **System supervisor and hard reset:** the IC integrates hardware watchdog monitoring, power-loss reset circuitry, and configurable GPIO/button controller logic capable of detecting wake-up events and long-press power-cycle commands from physical push-buttons (SW2/SW3 in Figure 11).
- **Zephyr driver architecture:** in firmware the nPM1300 is integrated into the Zephyr ecosystem via native multi-function device (MFD), regulator, and charger drivers (CONFIG_REGULATOR_CONFIG_CHARGER_NPM1300). The hardware layout is defined in the Devicetree, allowing Zephyr's power-management subsystem to throttle or disable voltage rails, handle charger interrupts, and transition the system into low-power sleep states when idle.

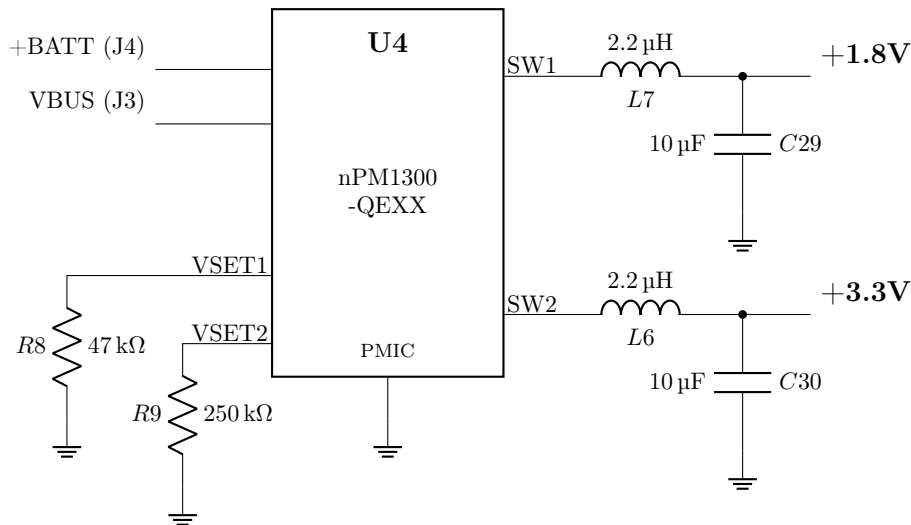


Figure 7: Power tree. A single-cell Li-Po (J4) and the USB VBUS rail (J3) feed the nPM1300 PMIC (U4). BUCK1 (L7 + C29) generates the 1.8 V core/IO rail; BUCK2 (L6 + C30) generates the 3.3 V peripheral rail. R8 and R9 program the two buck output voltages.

3 System design overview

This section describes what the system does and how the different functional blocks connect to each other. The lower-level implementation details are covered later in Section 5.

3.1 Requirements

We derived our hardware and software requirements directly from the main project objective:

ID	Requirement	Rationale / acceptance
R1	Wrist-wearable form factor	Board must fit a plausible watch case; Board radius: 22.9856 mm
R2	Single-cell Li-Po operation with USB charging	Charging and power path handled on-board, no external charger
R3	Days of battery life at typical duty cycle	Switching conversion throughout; MCU idle in deep sleep
R4	BLE connectivity to a phone	Standard GATT services; battery level notified to central
R5	Motion sensing with step counting, without continuous host polling	Met for the idle case: the sensor's wake-up detector raises an interrupt and the host sleeps until then. Step detection itself remains host-side but runs only while the wrist is moving (Section 5)
R6	Colour display with touch input	SPI display + capacitive touch controller
R7	Non-volatile storage for settings and logs	External QSPI flash with a file system
R8	Documented expansion port	Four contacts, ≤ 2 signal lines, 3.3 V module supply
R9	Runtime discovery of unknown modules	Host firmware must not need prior knowledge of a module
R10	Robustness against a faulty or hostile module	No module-supplied value may be used before validation; a jammed bus must be recoverable
R11	Firmware portable between the DK and the custom board	Devicetree overlay only; no application-code changes

Table 2: System requirements. R8–R10 are specific to this project's contribution and are addressed in Section 4.

3.2 System architecture

As shown in our block diagram (Figure 8), power comes from either the battery or the USB pogo header and goes into the PMIC. The PMIC converts this into two main rails: a 1.8 V digital rail and a 3.3 V peripheral rail. All the on-board digital logic runs at 1.8 V. Everything that crosses the enclosure boundary runs at 3.3 V. We used a bidirectional level shifter to connect the 1.8 V I²C bus from the SoC to the 3.3 V expansion port, which is what makes the expansion port usable by ordinary 3.3 V hardware.

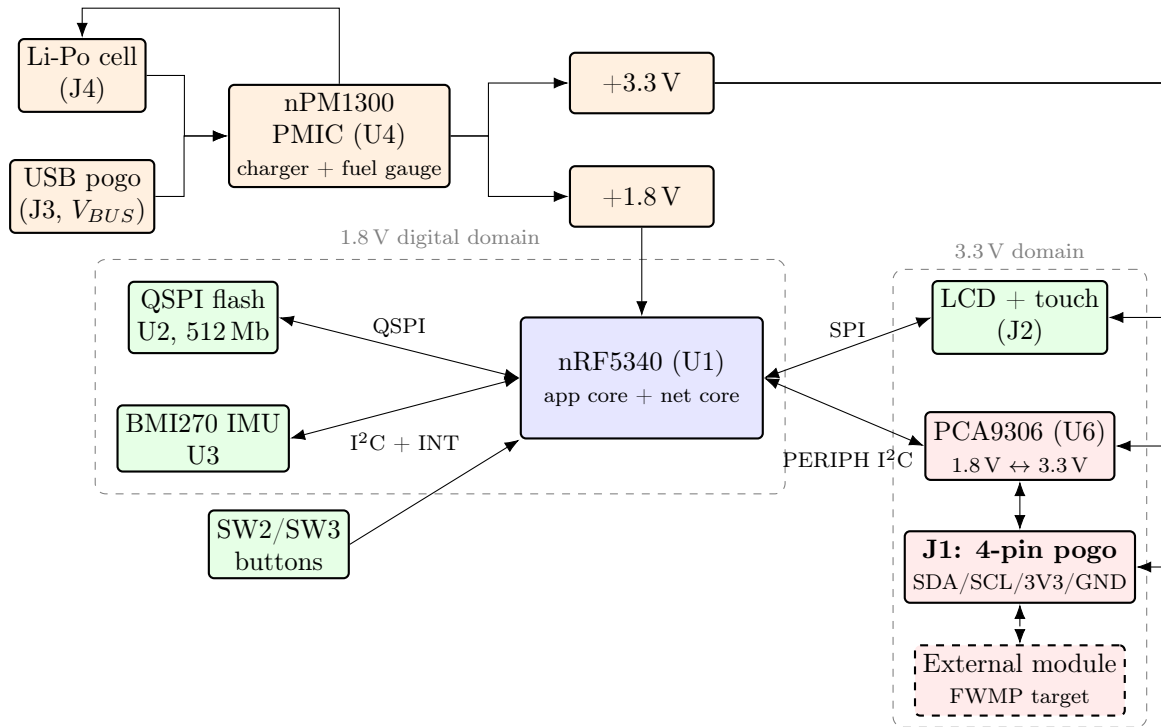


Figure 8: FinWatch system architecture. Power (orange) enters from the battery or the USB pogo header and is converted by the PMIC into the 1.8 V digital rail and the 3.3 V peripheral rail. On-board peripherals (green) attach directly to the SoC; everything crossing the enclosure boundary (red) does so at 3.3 V through the level shifter, including the expansion port J1 and any attached module.

3.3 Component selection

We selected every major part by comparing it against alternatives, and we summarized our reasoning in Table 3. However, it’s worth noting that two of these choices were eventually forced by component availability rather than pure technical merit, which we discuss later in Section 6.

Function	Chosen part	Alternatives	Reason for choice
SoC	nRF5340-QKxx, bare IC (U1)	nRF52840, ESP32-C3, STM32WB; or the same SoC as a pre-certified module (e.g. u-blox NORA-B10)	Dual core isolates the BLE stack from application timing; best-in-class sleep current; first-class Zephyr support. The bare IC rather than a module: smaller, cheaper, and it places the RF front end and fine-pitch escape inside the project’s scope — at the cost of owning the RF design and any certification (Section 8)
PMIC	nPM1300 (U4)	Discrete buck + MCP73831 charger + supervisor	One package replaces four; integrated fuel gauge; native Zephyr drivers; same vendor as the SoC
IMU	BMI270 (U3)	LSM6DSOX (original choice), ICM-42688	Operates from the 1.8 V rail; on-sensor pedometer, FSM, and Machine Learning Core can keep the MCU asleep; available when the LSM6DSOX was not (Section 6)
Flash	MX25U51245G (U2)	Internal 1 MB only; smaller SPI NOR	512 Mbit at 1.8 V, QSPI-capable, enough for fonts, assets, and logs
Level shifter	PCA9306 (U6)	Discrete MOSFET shifters; TXS0102	Purpose-built for I ² C; passes open-drain semantics correctly in both directions
Display	Waveshare LCD + capacitive touch (J2)	Bare panel + separate controller	Single FPC connector, known-good driver, reduces mechanical risk
Expansion	4-pin pogo (J1)	Board-to-board connector; USB-C alt mode	Sealed-surface contacts, no cavity through the case, cheap to implement on the module side

Table 3: Component selection and the alternatives considered.

3.4 Power budget

Table 4 shows our design-time power budget for each rail. We used this budget as the baseline for our battery-life estimates in Section 7, where we eventually replace these initial estimates with our actual measured figures.

Rail	Load	Typ. active	Typ. idle	Source
1.8 V	nRF5340 app + net core (BLE connected)	[TODO:]	[TODO:]	datasheet /
1.8 V	BMI270, accel only (embedded pedometer)	[TODO:]	[TODO:]	datasheet
1.8 V	BMI270, accel + gyro high-performance	[TODO:]	-	datasheet
1.8 V	MX25U51245G QSPI flash	[TODO:]	<10 μ A	datasheet (c
3.3 V	LCD + backlight	[TODO:]	0 (off)	measured
3.3 V	Touch controller	[TODO:]	[TODO:]	datasheet
3.3 V	Expansion module (budgeted)	[TODO: max mA to specify]		- design decis
Estimated system average		[TODO:]	[TODO:]	

Table 4: Power budget by rail. The expansion allowance is a specification decision rather than a measurement: it is the current a compliant module may draw, and it bounds the worst-case battery life the platform can promise.

3.5 Firmware architecture

We organized the firmware into nine subsystems. Each subsystem handles a specific piece of hardware and exposes a minimal C interface, along with a set of Zephyr threads running at fixed priorities. Table 5 lists the threads we used; keep in mind that lower numbers mean higher priority.

Thread	Priority	Stack	Responsibility
Display / LVGL	7	[TODO:]	Renders the UI and services the touch input device
Sensor	8	2048 B	Blocks on the BMI270 INT1 wake-up interrupt; samples at 10 Hz only inside an active window, detects steps, fires the motion callback
Update	9	1024 B	1 Hz watch-face refresh: time, steps; battery every 10 s
Expansion	10	1536 B	Presence polling, enumeration, input/output reports

Table 5: Firmware thread model. The expansion port is deliberately the lowest-priority thread on the system: a module must never be able to delay the display, the sensor, or the watch face by holding the bus.

The way we ordered these priorities was a deliberate design choice based on system safety. We set the expansion thread to run at the absolute lowest priority in the system. This means that if an attached module responds slowly or holds the clock line indefinitely, it only ruins its own responsiveness. Since the BLE stack also runs on a separate physical core, no misbehaviour

of an external module on the bus can freeze the user interface or drop the radio link. We also made the startup process fault-tolerant: inside `main()`, if any subsystem except the display fails to initialize, we log a warning and let the boot process continue. The expansion port is fundamentally optional hardware, so it must never prevent the watch from booting.

3.6 Use cases

Use case 1 - everyday wear. When no module is attached, the main MCU sleeps. The IMU watches for motion on its own. When the user raises their wrist, the sensor asserts an interrupt, the display wakes up, and the sensor thread samples at 10 Hz to count steps. A background thread updates the time and step count every second, and checks the battery every 10 s. The expansion thread just probes the I²C bus once a second and finds nothing.

Use case 2 - attaching a module. When the user clips a module onto the pogo contacts, the watch detects it after two consecutive successful probes. The host reads the module's descriptor, validates it, and finds out its name, its report sizes, and how often it wants to be polled. A new screen appears in the UI to show the module's data. If the user switches to a different screen, the polling rate backs off from 50 ms to 250 ms to save battery. If the module is pulled off, three consecutive failed probes trigger the teardown process. None of this requires the watch firmware to know anything about the specific module beforehand.

4 The FinWatch Module Protocol (FWMP)

This section breaks down our main contribution to the project: the protocol that lets any third-party hardware become a working part of a sealed smartwatch.

The protocol is built on three layers – electrical, logical, and behavioral.

Our guiding design principle across all three layers was simple: the watch must be able to accept a completely unknown module from an untrusted source, without needing any prior firmware updates, and without any fault in that module’s data or bus behavior being able to crash the watch.

4.1 The electrical contract

For the expansion port, we went with connector J1, which is a four-contact pogo-pin interface located on the outside of the watch case. Table 6 details the electrical layout of these pins.

Pin	Signal	Direction	Notes
1	+3V3	Watch → module	From PMIC BUCK2. Module current is bounded by the platform budget ([TODO: specify mA]); exceeding it is a specification violation
2	SDA	Bidirectional	Open drain, 3.3 V logic. Pulled up on the watch side by R4 (4.7 kΩ)
3	SCL	Watch → module	Open drain, 3.3 V logic, watch is always the bus controller. Pull-up R5 (4.7 kΩ)
4	GND	-	

Table 6: FWMP electrical interface (connector J1).

Choosing exactly four contacts had three major consequences that shaped the rest of our design:

- **Pull-ups belong to the watch.** We placed the I^2C pull-up resistors (R2/R3 on the 1.8 V side) inside the watch. A compliant external module must add its own pull-ups, the decision to not add pull ups on the watch side was intentional, because many boards that can be used to implement a module will have their own pull-up requirements(Raspberry Pi, Arduino).
- **There is no interrupt line.** Adding a fifth contact would have given us event-driven notifications, but we decided against it to save enclosure space and avoid mechanical alignment issues. Instead, we trade power for contacts: the watch discovers module presence and data availability strictly by polling.
- **The domain crossing is mandatory.** The nRF5340’s I/O pins run at 1.8 V, but almost all hobbyist sensors need 3.3 V. We added a PCA9306 level shifter between the SoC and the J1 connector. This costs us one extra component and a pair of pull-ups, but it makes the requirement that the port can talk to standard 3.3 V I^2C hardware actually work.

4.2 The logical contract: register map

We set the module to act as an I²C target at a fixed 7-bit address of 0x42.

We chose this specific address because it sits safely outside the reserved I²C ranges (0x00–0x07 and 0x78–0x7F).

Since this I²C bus is dedicated entirely to the expansion port and only connects to one module at a time, a fixed address is all we need.

This avoids the added complexity of implementing dynamic address assignment on a bus that doesn't have out-of-band signaling.

The module exposes a flat register space, which we read using the standard EEPROM method: write the offset, send a repeated START, and read the bytes.

As shown in the mapping below, the lower registers store the static descriptor, while the higher registers handle the runtime input and output reports.

Offset	Name	Size	Meaning
<i>Descriptor header (fixed, 10 bytes)</i>			
0x00	MAGIC0	1	0x46 ('F')
0x01	MAGIC1	1	0x57 ('W')
0x02	PROTO_VER	1	Protocol version; 0x01 for FWMP v1
0x03	DESC_LEN	1	Total descriptor length: header + TLVs + CRC
0x04	VID	2	Vendor ID, little-endian
0x06	PID	2	Product ID, little-endian
0x08	CAPS	1	Capability bitmask (Table 8)
0x09	N_TLV	1	Number of TLV entries that follow
0x0A	TLV area	var	TLV entries, then a trailing CRC-16
<i>Runtime registers</i>			
0x80	STATUS	1	Status bits (Table 8)
0x81	SEQ	1	Incremented by the module on each new input report
0x82	INPUT	var	Input report, length declared in the descriptor
0xC0	OUTPUT	var	Output report written by the host

Table 7: FWMP v1 register map. STATUS and SEQ are adjacent by design so that a single two-byte read answers both “is there new data?” and “is it data I have already seen?”.

Field	Bit	Meaning
CAPS	0 (CAP_INPUT)	Module produces input reports
CAPS	1 (CAP_OUTPUT)	Module accepts output reports
STATUS	0 (DATA_READY)	An unread input report is waiting
STATUS	1 (ERROR)	Module-side error condition
STATUS	2 (BUSY)	Module cannot service a transaction now

Table 8: Capability and status bit definitions.

4.3 The descriptor: self-description through TLVs

The fixed header tells the host that a module is connected and which protocol version it uses, but it doesn't tell us what the module actually is.

To solve this, we used a sequence of Type-Length-Value (TLV) entries located between the header and the trailing CRC.

Each TLV entry consists of one type byte, one length byte, and the actual payload bytes.

Type	Name	Payload	Meaning
0x01	NAME	ASCII, ≤ 24 B	Human-readable module name, not NUL-terminated on the wire
0x02	INPUT_REPORT	3 B	[len] [poll_hint_ms:2 LE] - report size and requested poll interval
0x03	OUTPUT_REPORT	1 B	[len] - output report size
0x04	ICON_ID	1 B	Index into the watch's built-in icon set

Table 9: FWMP v1 TLV types. A host that encounters an unknown type skips it rather than rejecting the descriptor, which is what makes the format forward-extensible: a v2 module carrying new TLVs still enumerates correctly on a v1 watch.

This TLV structure is what makes our FWMP different from other ecosystems like Qwiic, which we discussed in Section 1.2.

Those ecosystems just standardize the physical connector, meaning the host still needs a specific driver compiled for every new device.

In our design, the descriptor carries enough information – like the module's name, the sizes of its data reports, its preferred polling rate, and an icon ID – so that a completely unknown module becomes useful the second it's plugged in.

The watch can show the module to the user and start exchanging data without us having to write a single line of module-specific code. We also made a specific design choice for forward compatibility: if the host sees a TLV type it doesn't recognize, it just skips it instead of rejecting the whole descriptor.

We borrowed this idea from how USB HID descriptors work. If we had used strict rejection instead, releasing a v2 module in the future would instantly break every deployed v1 watch.

4.4 Enumeration and the trust boundary

We designed the system under the assumption that everything the module sends is untrusted. Since it's third-party hardware attached by the user, it could easily be defective or even malicious. That's why our enumeration process acts as a strict validation pipeline where we check every value before using it:

1. We start by reading just the 10-byte fixed header, because we don't know the full length yet.
2. We verify the magic bytes (0x46, 0x57) to ensure it's actually an FWMP module.
3. We check the protocol version and reject unsupported versions immediately.

4. We bound the declared descriptor length. This is the most critical check — it prevents a fake length byte from causing a buffer overrun on the host side, and we cap it at 128 bytes.
5. We read the rest of the descriptor and verify the CRC-16 checksum to catch corrupted data.
6. We parse the TLVs while keeping strict bounds checking. If a payload length tries to run past the end of the descriptor, we abort the process.
7. Only after passing all these steps do we trust the descriptor, load the info structure, and move the module to the ACTIVE state.

To make this even safer, we store the TLV cursors as 16-bit integers, so future size increases won't cause silent wraparound bugs.

Also, if a module fails validation, it gets placed in an ERROR state and we stop trying to read it until it's physically removed, preventing infinite enumeration loops.

4.5 The behavioural contract: state machine

Figure 9 shows our host-side state machine.

Since we don't have a dedicated interrupt line, we check if a module is present by seeing if it ACKs its I²C address.

Physical pogo contacts tend to bounce, so we debounce this in software: we require two consecutive successful probes to register an attachment, but three consecutive NACKs to register a detachment.

We made this asymmetry on purpose. Registering an attachment too early just leads to a failed enumeration, but dropping a connection too early ruins an active session and throws away state.

Because of that, we require stronger proof for removing a module than for plugging one in.

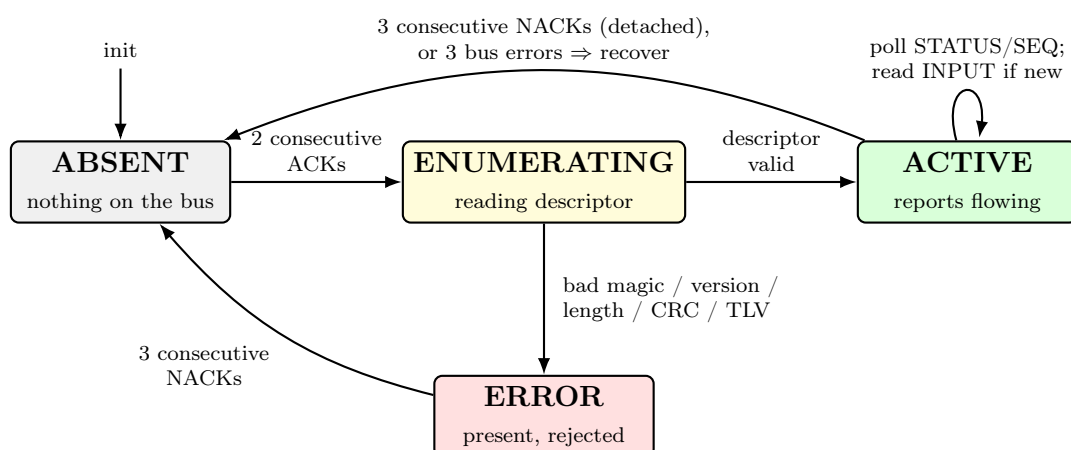


Figure 9: Host-side enumeration state machine. Attachment requires two consecutive address ACKs and detachment three consecutive NACKs, debouncing pogo-contact bounce in both directions. A module whose descriptor fails validation lands in ERROR and is not retried until it is physically removed, so a broken module cannot trap the host in a re-enumeration loop.

Once the module enters the ACTIVE state, every polling cycle reads the **STATUS** and **SEQ** registers together in a single two-byte transaction.

We only fetch a new input report if the **DATA_READY** bit is set and the sequence number (**SEQ**) is different from the last one we saw.

This way, if a module has nothing new to say, it only costs us two bytes on the bus instead of reading a full report.

We copy these reports into a buffer protected by a mutex, since the UI thread reads them at the exact same time the expansion thread is writing them.

We also count bus errors separately from normal **I²C** NACKs.

If we hit three consecutive bus errors while active, we trigger Zephyr's `i2c_recover_bus()` function.

This sends clock pulses to recover a target that is stuck holding the SDA line low, and then we reset the port to ABSENT.

This directly answers requirement R10 and handles the open-drain hazard we mentioned in Section 2: if someone rips a module off right in the middle of a transaction, it jams the bus, so our port needs to be able to un-jam itself without forcing the user to reboot the watch.

4.6 Adaptive polling policy

Because we rely on polling to detect both module presence and incoming data, the polling interval dictates our entire power-versus-latency trade-off for the expansion port. We handle this in FWMP by using three different polling regimes, which are summarized below:

Regime	Interval	When
Absent	1000 ms	No module attached. One zero-length transaction per second is the standing cost of supporting expansion at all
Background	250 ms	Module attached but its screen is not visible
Foreground	50 ms	User is looking at the module screen; bounded below by the host, and lengthened to the module's <code>poll_hint_ms</code> if the module asks for something slower

Table 10: Adaptive poll intervals. The host honours a module's requested rate only in the direction that saves power: a module may ask to be polled *less* often than 50 ms, never more.

The way we handle `poll_hint_ms` asymmetrically is another example of our trust boundary. Since this hint is data supplied by the module, we only let it influence the host's behavior when a malicious or buggy value is harmless.

If a module claims it needs to be polled every single millisecond, it would completely drain the watch's battery, so we clamp that request.

On the other hand, if a module asks to be polled every two seconds, honoring that request costs us nothing.

4.7 Writing a compliant module

The full contract for building a third-party module is short enough that we can state it all in one place:

- M1.** Act as an `I2C` target at 7-bit address 0x42, 3.3 V logic, up to 400 kHz.
- M2.** Do not fit pull-up resistors; the watch owns them.
- M3.** Draw no more than the platform current allowance from pin 1 (**[TODO: specify]**).
- M4.** Implement the register map of Table 7, readable with the write-offset/repeated-START idiom.
- M5.** Serve a descriptor whose magic is 'FW', whose version is 0x01, whose `DESC_LEN` is truthful and ≤ 128 , and whose final two bytes are a CRC-16/CCITT (init 0xFFFF) over all preceding bytes.
- M6.** Declare at least a `NAME` TLV, plus `INPUT_REPORT` and/or `OUTPUT_REPORT` matching the `CAPS` bits actually set.
- M7.** Increment `SEQ` on every new input report and set `DATA_READY` in `STATUS`; clear it when the report has been read.

M8. Tolerate being polled at any rate between 50 ms and 1 s, and tolerate power being removed at any instant.

Nothing in this checklist requires anything more powerful than a basic 8-bit microcontroller with an [I²C](#) peripheral.

That was exactly our design target: we wanted the barrier to entry for making a FinWatch module to be a weekend project, not a full-blown development contract.

You can find our reference module design built against this exact checklist in [Appendix C](#).

4.8 Summary

To wrap up, FWMP v1 meets our expansion requirements (R8–R10) by using a four-contact interface, a fixed address, and a CRC-protected self-describing descriptor.

The key takeaway is that an unknown module becomes completely usable without any host-side code, while our strict bounds-checking ensures the watch remains safe from malformed or malicious module data.

5 Implementation

5.1 Schematic capture

Once we finalized the system design, we moved to KiCad to capture the schematic.

We wired up all the components to their correct operating voltages and matched every part to its appropriate protocol interface.

The full design is saved in the `watch_thing_alt.kicad_sch` file.

Figures 7, 3, 10 and 11 break down the schematic into functional blocks – the PMIC power tree, the nRF5340 core with its RF front end, the peripherals, and the external pogo-pin I/O with its level shifter.

The official KiCad exports are attached in Appendix B.

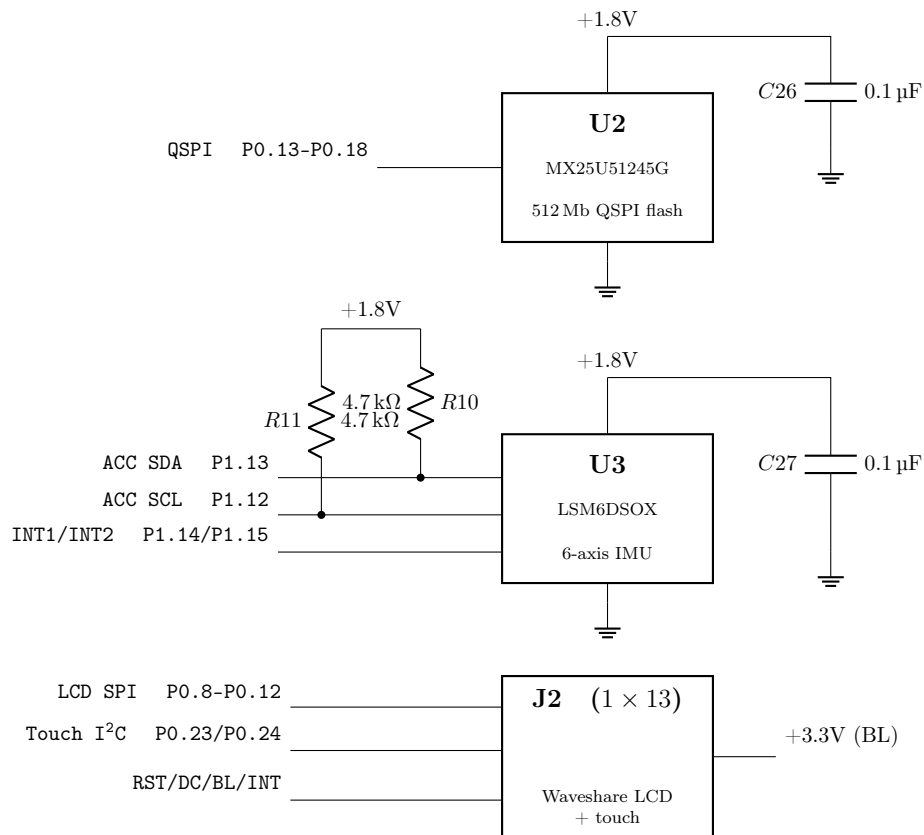


Figure 10: Peripherals. The QSPI flash (U2) and LSM6DSOX IMU (U3) run from the 1.8 V rail, each with a local 0.1 μ F decoupling capacitor; R10/R11 pull up the accelerometer I²C bus. The SPI display and its capacitive-touch controller share connector J2.

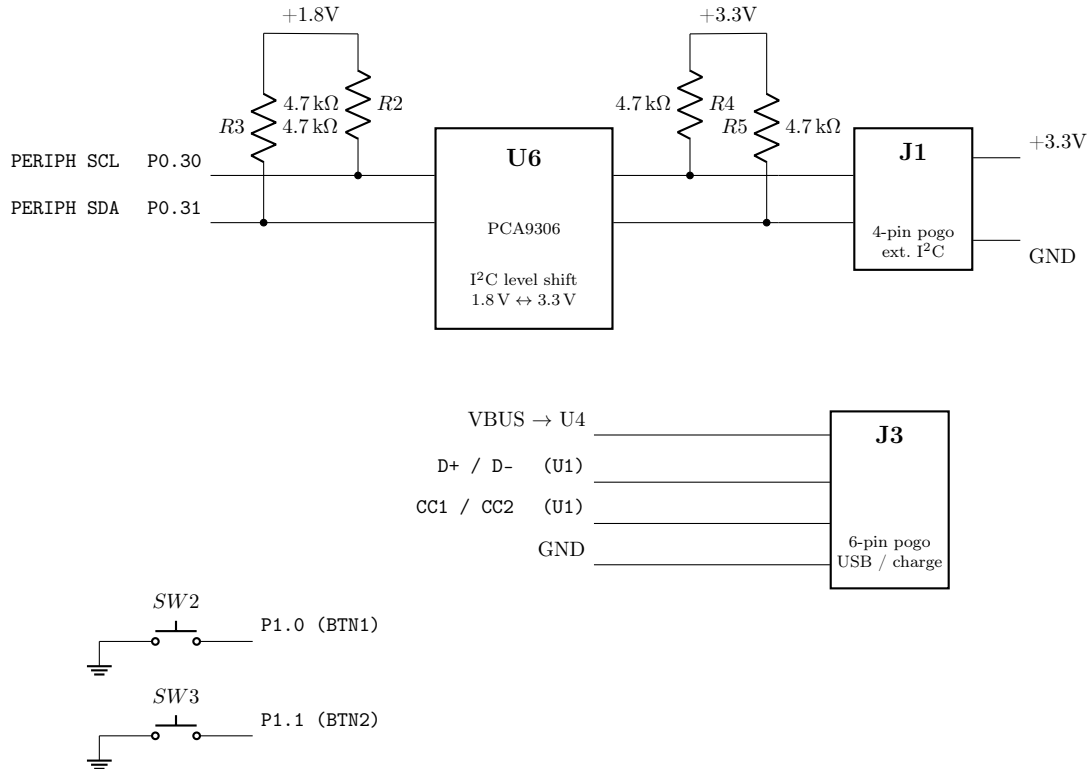


Figure 11: External I/O. The PCA9306 (U6) translates the nRF5340 1.8 V PERIPH I²C bus to 3.3 V on the J1 pogo header, with pull-ups R2/R3 on the low side and R4/R5 on the high side - the electrical contract of Table 6. J3 carries USB VBUS, D+/D- and the USB-C CC1/CC2 lines used for charging. SW2/SW3 are the user push-buttons on P1.0/P1.1.

5.2 PCB layout

After finishing the schematic, we tackled the PCB layout.

We strictly applied the design principles we discussed back in Section 2.

We placed the decoupling capacitors as close as physically possible to their respective supply pins, kept the switching loops of the buck converters tight around the PMIC, ensured an uninterrupted ground return path under every signal, and kept all copper out of the antenna region.

Parameter	Value
Board outline	Circular, 45.97 mm diameter
Layer count	4 (F_Cu, In1_Cu, In2_Cu, B_Cu)
Board thickness	1.0 mm, FR-4
Copper weight	1 oz outer, 0.5 oz inner
Surface finish	ENIG (immersion gold)
Solder mask / silkscreen	Green / white
Minimum drill	0.15 mm (27 holes)
Drill classes	0.15, 0.20, 0.30, 0.50 mm plated; 0.90, 2.10 mm non-plated
Total holes	170 (164 plated, 6 non-plated)
Via treatment	Epoxy filled and capped (IPC-4761 Type VII)
Component count	30 BOM lines, 88 placements
Manufacturer / process	JLCPCB, 4-layer FR-4 with resin via fill
Panelisation	71.97 mm square panel, 13 mm process rails

Table 11: PCB physical parameters, as confirmed against the manufacturer’s CAM output.

5.2.1 Via-in-pad and the cost of fine-pitch escape routing

The nRF5340 comes in an aQFN-94 package, which squeezes 94 pads onto a 0.4 mm grid inside a 7×7 mm body with a central thermal pad.

Routing all these signals out was easily the hardest layout challenge on the board, and it’s what ultimately dictated our layer count, minimum drill size, and overall manufacturing process.

Why the conventional escape fails. The standard *dog-bone* technique – routing a short trace from the pad to a via placed in the empty space between the pads – simply fails here.

At a 0.4 mm pitch, that space just does not exist.

Even using a tiny 0.15 mm drill with a minimum annular ring and standard clearance rules, there is no physical width left for the escape trace.

We could only escape the outer row or two, leaving the inner rows completely unreachable.

Via-in-pad. The only working solution is placing the via directly inside the component pad, dropping the signal straight to an inner layer so no escape trace is needed.

We placed 25 vias (0.15 mm) exactly on the package’s 0.4 mm grid inside the pads of U1.

Every single one sits perfectly dead-center in its pad, perfectly aligned with the solder-paste aperture and solder-mask opening.

However, via-in-pad is not free – you just pay for it in manufacturing instead of layout. An open via is basically a hole beneath molten solder.

During reflow, capillary action will suck the solder down the barrel and away from the joint. This leaves an unpredictable, starved, or voided connection under a package that we can’t even inspect without an X-ray machine.

To prevent this, the via has to be filled and leveled before the board is ever assembled.

The filled-and-capped process. Our board required the IPC-4761 Type VII filled-and-capped treatment, which forces the fabricator to inject several extra steps right in the middle of normal production:

1. Drill and plate the vias as usual to form the copper barrel.

2. Fill each barrel with a non-conductive epoxy resin under a vacuum, making absolutely sure no air gets trapped.
3. Cure the resin.
4. Planarize the board by grinding the surface flush, removing any excess resin that sticks out above the copper.
5. Plate a solid copper “cap” over the filled via to close it up and restore a perfectly flat, solderable pad.
6. Continue with the standard outer-layer imaging, solder mask, and surface finish.

The consequences of this process for a project of our scale are worth spelling out, mainly because they are completely invisible in the layout files:

- **Cost.** Via filling comes with a per-order surcharge, and the 0.15 mm drills triggered a “small-hole” fee as well. We were charged for both on this order.
- **Lead time.** Because these four extra steps (fill, cure, planarize, cap) happen in the middle of fabrication, you cannot expedite the board past them.
- **Supplier choice.** Not every low-cost board house offers resin fill with cap plating. Requiring it immediately narrowed down our options before we even asked for quotes, which tied directly into the procurement headaches we discuss in Section 6.
- **Failure modes move upstream.** If the manufacturer leaves air or moisture in the via, it will outgas during reflow and literally blow out the joint above it. If they over-grind or under-plate the cap, it leaves a dimple that ruins the pad’s coplanarity. We cannot see or fix either of these issues after assembly, so our design relies entirely on the fabricator’s process control rather than anything we can verify in the lab.

It is worth noting that none of this was strictly forced by the application itself.

If we had used a pre-certified module with the same SoC, we would have had coarse-pitch pads that route easily on two layers without any via-in-pad at all (and comparable projects take exactly that route, as discussed in Section 8).

We consciously paid the price of using bare silicon to save board area, reduce unit costs, and take full ownership of the RF design and escape routing.

The general lesson here is that choosing a 0.4 mm-pitch package isn’t just a routing decision – it transfers a big chunk of the board’s reliability from the layout phase, where we can check it, to the fabricator’s floor, where we can’t.

Having chosen bare silicon for a board that has to fit inside a watch case, no coarser-pitch package was a realistic alternative, but we made the trade-off knowingly.

This is exactly why we went through the manufacturer’s CAM files hole-by-hole before production.

In addition, we made a few important layout decisions that are not obvious from the schematic.

1. We placed the PMIC and its two inductors in a tight cluster in the middle of the board, so the switching loops are as small as possible and the EMI is contained.
2. We placed the IMU near the board center so its axes are consistent regardless of how the watch is worn.
3. We placed the antenna at the board edge with a keepout zone around it, and routed the RF match as a controlled-impedance line.
4. We routed the 1.8 V and GND power planes on the inner layers to ensure there was a short return path for the high-frequency signals.
5. We used a 4-layer stackup to allow for better routing of the high-density signals and to provide a solid ground reference for the RF signals.

5.3 Mechanical design

[TODO: Fusion enclosure: case dimensions, how the pogo contacts are exposed without breaching the sealed surface, display window, button placement, strap attachment. Include a render.]

5.4 Firmware

We wrote the firmware targeting the nRF5340 application core using the Zephyr RTOS and the nRF Connect SDK. To keep things organized, we divided the code into nine separate subsystems. Each subsystem sits in its own directory under `src/` and exposes a minimal header file to the rest of the application.

Subsystem	LOC	Responsibility
expansion/	915	FWMP host: transport, protocol, state machine, mock backend
display/	593	LVGL setup, watch face, module screen, test screen
sensor/	395	LSM6DSOX wake-up interrupt, active-window sampling, step detection
ble/	294	GATT services, advertising, battery notification
main.c	178	Subsystem init, button handling, 1 Hz update thread
rtc/	115	Timekeeping
storage/	110	File system on QSPI flash, settings persistence
buttons/	78	Debounce, short/long press classification
touch/	73	Capacitive touch controller, LVGL input device
pmic/	58	nPM1300 battery percentage and charge state
Total	2 809	

Table 12: Firmware size by subsystem (lines of C, headers included). The expansion port is the largest single subsystem in the project - roughly a third of the codebase - which reflects that it is the contribution rather than the plumbing.

Hardware abstraction. Following the design principles we mentioned in Section 2, we didn't hardcode any GPIO pins, bus frequencies, or I²C addresses of on-board devices into our C code. Everything is resolved cleanly from the Devicetree.

However, we made one deliberate exception: the expansion module itself.

Since the module is hot-plugged hardware that might not even be attached when the watch boots, it doesn't get a Devicetree node.

Declaring a child node for a device that is usually absent is a misuse of the Devicetree model. Instead, our transport layer takes the I²C bus controller (i2c3) from the Devicetree and addresses 0x42 directly.

Transport abstraction and the mock backend. We split the expansion subsystem so our protocol logic never has to touch Zephyr's I²C API directly.

We declared five basic operations (init, probe, read, write, recover) and built two backends: the real I²C hardware backend, and a mock backend.

The mock backend simulates a "Test Dial" module with a rotating input report and can intentionally inject six different classes of fault on demand – five kinds of malformed descriptor and a bus error.

Building this mock allowed us to fully test the protocol logic without any physical module hardware, and it serves as the foundation for our fault-injection tests in Section 7.

Interrupt-driven motion wake. To satisfy requirement R5, we had to make sure the host doesn't continuously poll the IMU.

We shifted the responsibility of detecting motion from the CPU directly to the sensor.

The BMI270 has a built-in wake-up function that compares the acceleration slope against a programmable threshold and asserts the INT1 pin when that threshold is crossed.

We configure this directly over I²C because the standard Zephyr driver only gives us a "data-ready" trigger.

Using a data-ready trigger would wake the CPU for every single sample, which completely defeats the purpose of putting the MCU to sleep.

Therefore, we left the Zephyr trigger unconfigured so the driver doesn't claim the interrupt line, and let our custom sensor subsystem take full ownership of it.

This setup creates a clean, two-state control flow:

- **Idle.** The sensor thread blocks indefinitely on a semaphore. It consumes zero CPU time and generates absolutely no timer wake-ups, while the IMU automatically drops into a 12.5 Hz low-power mode.
- **Active.** When the INT1 pin fires, the interrupt service routine (ISR) releases the semaphore. The thread wakes up, reads the WAKE_UP_SRC register to confirm a genuine motion event, wakes the display, and starts sampling at 10 Hz for step detection. The active window closes 3s after the last motion event (or immediately if the sensor reports inactivity), and the thread goes right back to sleep.

We don't lose any step data by only sampling inside this active window, because steps literally only happen when the wrist is moving.

We configured the interrupt as latched, meaning we have to acknowledge it by reading the source register.

If we miss an acknowledgment, the INT1 line stays high and silently kills the entire wake-up

path.

To prevent this, we read the source register on every single event and every time we exit the active window.

We also built in a fail-safe: if the interrupt fails to initialize (like if the pin isn't wired properly on a bench setup), the firmware logs a warning and falls back to continuous 10 Hz polling.

This ensures the step counter still works, but it degrades the battery life. **[TODO: Verify on hardware. This path has not been run on a physical BMI270 or LSM6DSOX yet. We need to: confirm that INT1 asserts on wrist raise, that WAKE_UP_SRC reports WU_IA, that the active window closes as expected, and that repeated wake events continue to produce edges (the latched-interrupt acknowledgement above is the failure mode to watch for). Tune WAKE_UP_THS_LSB (currently 2, ≈ 125 mg at $\pm 4g$) against real wrist motion.]**

Toolchain and dependencies.

- Zephyr RTOS **[TODO: version]** via nRF Connect SDK **[TODO: version]**
- Arm GNU toolchain **[TODO: version]**, built with `west`
- LVGL **[TODO: version]** for the user interface
- KiCad **[TODO: version]** for schematic and PCB
- Autodesk Fusion **[TODO: version]** for the enclosure

6 Design challenges and revisions

This section goes over the main problems we ran into that ended up shaping the final design. We are explicitly documenting our Revision A defect here, not because it is flattering, but because the debugging process and the checklist it generated are actual, valuable results of this project.

6.1 Revision A defect: no debug access

Symptom. We manufactured and assembled the first PCB revision. The power rails came up correctly, but the board was completely unprogrammable – we couldn’t load firmware or debug it at all.

Root cause. We simply forgot to route the nRF5340’s SWDIO and SWDCLK pins in the schematic. Since a blank nRF5340 has no bootloader, the Serial Wire Debug (SWD) interface is the only way to flash the very first image. Omitting it essentially rendered the board permanently unprogrammable. This mistake slipped right past our design review because SWD isn’t a functional net for any other component on the board. An Electrical Rules Check (ERC) only verifies the connections you actually drew; it cannot warn you about connections that should exist but don’t.

Fix. In Revision B, we routed the three debug signals out to 1.0×1.0 mm test pads on the board surface: SWDIO to TP4, SWDCLK to TP3, and nRESET to TP2. We used test pads instead of a physical header to save board area and enclosure volume, which is critical for a watch. We also made sure to include the nRESET line. If faulty firmware locks up the core, a simple two-pad SWD setup won’t let the debugger halt the device, so you absolutely need the reset line to make the debug interface robust.

Lesson and process change. This failure directly led us to create a strict pre-fabrication checklist (Section 6.7) to manually verify all the critical nets that automated rules checks miss.

6.2 Component availability

During the project, supply chain shortages affected both our PMIC and our IMU.

PMIC shortage. The nPM1300 PMIC we specified was out of stock at every distributor we checked, and the lead time quoted by the manufacturer was longer than our remaining project schedule. We had to switch to a different PMIC (the nPM1100) that was in stock and could be delivered on time.

Since the nPM1300 uses a unique package and pinout, we couldn’t just swap the part, we had to wait and check all of the leading distributors for stock, until we found one that could deliver the part in time.

Procurement as a design constraint. We learned firsthand that procurement is a strict design constraint.

Several suppliers refused to even quote us for the single-unit quantities a student project requires, and others quoted lead times longer than our remaining project schedule.

We even had one order canceled outright after it was placed due to an administrative error on the supplier's side, which cost us a further 14 weeks before we could arrange an alternative.

These issues had two direct consequences for our design.

First, the part we specified wasn't always the part we could obtain.

Second, availability stopped being just a procurement detail and became a core selection criterion alongside the technical specs in Table 3.

A part with a unique footprint and no compatible alternate carries a massive schedule risk.

This reality is invisible in the finished hardware, but it is a major difference between our student prototype and a commercial product backed by real purchasing power and supplier relationships.

Lesson. In a student project with a single fabrication opportunity, second-source availability deserves to be a primary selection criterion.

Our IMU substitution showed exactly what makes a second source genuinely usable rather than merely similar: because the replacement was pin-for-pin compatible, the board needed absolutely no changes.

Because our hardware dependencies were isolated in the Devicetree, the software change needed for the substitution itself was confined to an overlay node and a driver selection (the interrupt-driven wake path added later, Section 5.4, does program LSM6DSOX registers directly).

If we had hardcoded the sensor's address and register map into our C code, or used a part with a unique pinout, this exact same shortage would have forced us to do a full board re-spin.

6.3 The two-voltage-domain problem

The nRF5340 operates its I/O at 1.8 V, but essentially every peripheral a user might attach (and the display backlight) expects 3.3 V. We evaluated three options:

1. Run the entire board at 3.3 V. (Rejected: it raises the switching power of every digital transition on the board, and the MX25U51245G external flash is a 1.8 V-only part.)
2. Expose the expansion port at 1.8 V and require modules to match. (Rejected: makes the port useless for the target audience, since off-the-shelf sensor breakouts run at 3.3 V.)
3. Keep the digital domain at 1.8 V and translate at the boundary. (Chosen.)

We went with option 3. This cost us one PCA9306 level shifter and a second pair of pull-up resistors. The PCA9306 strictly relies on pull-ups on both sides to function, so the resistor pairs aren't redundant duplicates — they are required. This is exactly why we explicitly forbid external modules from incorporating their own pull-ups (rule M2 in Section 4).

6.4 No interrupt line on the expansion port

As discussed in Section 4, we rejected adding a fifth pogo contact to save mechanical space. The trade-off is that our purely polled port constantly burns idle current and has a worst-case attach latency of two poll intervals. Our adaptive polling policy (Table 10) is the mitigation for this.

6.5 Hot-plug on an open-drain bus

If a user unplugs a module mid-transaction, it can leave the SDA line held low, which jams the bus until it is recovered. This isn't hypothetical; it's exactly what can happen when physical contacts are disconnected unexpectedly. To handle this, the host explicitly counts consecutive bus errors separately from normal NACKs. If it hits three errors, it calls Zephyr's `i2c_recover_bus()` to send clock pulses and un-jam the line, resetting the port back to the ABSENT state without requiring a hard reboot.

6.6 Storage subsystem partition-ID collision

During our DK bring-up, mounting the file system caused a SecureFault instead of a graceful failure. The fixed-partition ID we used for our external QSPI flash collided with the DK's internal `storage_partition`. Since the external flash isn't populated on the DK, the mount should have just failed so the boot could continue. We temporarily disabled storage initialization in `main()` until we have the real flash hardware on Revision B.

This was added back to the final board.

6.7 Pre-fabrication design-review checklist

Derived from all the problems above, this is the checklist we now apply before releasing a board revision:

- C1.** Is SWD (SWDIO, SWDCLK, nRESET, reference ground, target voltage) brought out to an accessible connector or pad set?
- C2.** Does every power rail have a test point, and is there at least one ground clip point?
- C3.** Has ERC been run with no unexplained warnings, *and* has a human separately verified the nets ERC cannot know about (debug, straps, unused-pin termination)?
- C4.** Has DRC been run clean against the manufacturer's actual capabilities, not the default profile?
- C5.** Is every part on the BOM in stock, from at least two distributors, with a documented second source?
- C6.** For each critical part, is there a functionally equivalent alternate in the *same footprint*, so that a substitution costs a BOM line rather than a board revision?
- C7.** Have quoted lead times and minimum order quantities been checked against the remaining project schedule, with the order placed early enough to absorb one cancellation?
- C8.** Does every IC have local decoupling placed on the same layer and within a few millimetres of its supply pin?
- C9.** Is the antenna keepout free of copper on all layers, and is the RF match routed as a controlled-impedance line?
- C10.** Have the mechanical outline, connector positions, and mounting holes been checked against the enclosure model?

7 Verification and results

We verified our system on three main fronts, based on what hardware was available at the time. We ran functional verification of the firmware on the nRF5340-DK, electrical verification on our manufactured Revision A board, and fault-injection testing on the expansion protocol using our mock transport. For each experiment below, we outline the objective, the method, the results, and exactly what those results do and don't prove.

7.1 Experiment 1 - Functional verification on the nRF5340-DK

Objective. We wanted to prove that the complete firmware — including the display, touch, sensor, BLE, buttons, and FWMP host — works correctly regardless of the custom board.

Method. We compiled the firmware for the nRF5340-DK target and tested it manually against a smartphone acting as the BLE central. Because Zephyr abstracts the hardware into the Devicetree, we were able to run the exact same application code on the DK that we will eventually use on the custom board.

Results. [TODO: Fill in per subsystem: display renders watch face (yes/no), touch registers input, the LSM6DSOX reports acceleration and the host-side step detector increments on walking, motion wakes the screen, BLE advertises and a central connects and receives battery notifications, buttons produce short/long events. State which subsystems were not verifiable on the DK - external QSPI flash is not populated, so storage was disabled; the PMIC is absent, so battery percentage is not meaningful.]

Interpretation. This confirms that our application logic and FWMP host implementation are solid, even if it doesn't prove anything about the custom PCB itself. Being able to separate the software from the hardware allowed us to keep developing firmware in parallel with board fabrication.

7.2 Experiment 2 - Expansion protocol, nominal path

Objective. We needed to show that the watch can discover, enumerate, and interact with a module without having any module-specific code pre-compiled into the firmware.

Method. We enabled `CONFIG_FINWATCH_EXPANSION MOCK=y` to let the mock backend simulate plugging in a fake “Test Dial” module. This fake module reports a vendor ID of `0xABCD`, a product ID of `0x0001`, both `INPUT` and `OUTPUT` capabilities, a 4-byte input report, a 1-byte output report, and a 100 ms requested poll interval. We triggered the attach and detach events manually using `expansion_mock_set_attached()`.

Results. [TODO: Paste the log output showing: probe ACK, descriptor accepted, the module "Test Dial" vid=abcd pid=0001 caps=03 in=4 out=1 line, the module screen appearing, live report values changing, an output report being written and echoed back in byte 2 of the next input report, and the detach transition.]

Interpretation. The watch successfully displayed and exchanged data with a module that it didn't know existed at compile time, strictly by reading its descriptor. This proves the core claim we made in Section 4 works end-to-end.

7.3 Experiment 3 - Expansion protocol, fault injection

Objective. We wanted to verify that the host safely catches every single rejection path, ensuring that a malformed descriptor can never cause the watch to crash or behave incorrectly.

Method. We used the mock backend to artificially inject six different classes of faults via `expansion_mock_set_fault()`. The backend rebuilds the descriptor with the chosen fault applied, and the host tries to enumerate it normally.

Fault	What is corrupted	Expected	Observed
BAD_MAGIC	MAGICO set to 0x00	Reject, ERROR	[TODO:]
BAD_VERSION	PROTO_VER set to 0x99	Reject (-ENOTSUP)	[TODO:]
BAD_CRC	Trailing CRC inverted	Reject, ERROR	[TODO:]
OVERSIZE_LEN	DESC_LEN declared as 200 (> 128)	Reject before any read	[TODO:]
TLV_OVERRUN	INPUT_REPORT TLV claims a 200-byte payload	Reject, no out-of-bounds read	[TODO:]
BUS_ERROR	All transactions return -EIO	3 errors $\Rightarrow$ recover, $\rightarrow$ ABSENT	[TODO:]

Table 13: Fault-injection matrix for the FWMP host. Each row corresponds to one validation step in Section 4.4; together they cover every rejection branch in the enumeration path.

Interpretation. [TODO: After filling the observed column: state that all six faults are rejected without incorrect state transitions and without reads past buffer bounds, and note that the OVERSIZE_LEN and TLV_OVERRUN cases in particular are the ones that would be memory-safety failures in a naive implementation.]

7.4 Experiment 4 - Revision A electrical bring-up

Objective. We needed to verify that the power architecture on our manufactured Revision A board actually works as designed, since the missing debug access meant we couldn't test it with firmware.

Method. [TODO: Describe: bench supply or battery, what was probed, with what instrument.]

Measurement	Expected	Measured	Pass/Fail
BUCK1 output	1.8 V $\pm$ tolerance	[TODO:]	[TODO:]
BUCK2 output	3.3 V $\pm$ tolerance	[TODO:]	[TODO:]
BUCK1 ripple (p-p)	[TODO:]	[TODO:]	[TODO:]
BUCK2 ripple (p-p)	[TODO:]	[TODO:]	[TODO:]
Expansion port pin 1 (J1)	3.3 V	[TODO:]	[TODO:]
Charge current from V_{BUS}	[TODO:]	[TODO:]	[TODO:]
Quiescent current, idle	[TODO:]	[TODO:]	[TODO:]

Table 14: Revision-A electrical measurements.

Interpretation. [TODO: The power tree is the part of the board that could be validated without SWD access; say explicitly what this does and does not prove about the rest of the design.]

7.5 Experiment 5 - Battery life estimate

Objective. We wanted to estimate how long the watch can run on a single charge under a realistic daily duty cycle.

Method. We combined our measured per-rail current values with an assumed usage profile (display off and BLE connected most of the day, a set number of wake events per hour, and the expansion port either empty or attached).

Results. [TODO: Compute from Table 4 once measured. Give three figures: no module attached, module attached in background polling, module attached with screen in foreground. The delta between the first two is the honest cost of the expansion feature.]

7.6 Experiment 6 - Cost of interrupt-driven wake

Objective. We wanted to put a real number on how much current we save by using the sensor's wake-up interrupt instead of continuously polling the IMU at 10 Hz.

Method. Since our firmware includes both methods, we forced the fallback polling path by deconfiguring the interrupt, giving us a perfect A/B test on the exact same hardware and code. We measured the average current under three conditions: device sitting still with the display off, device sitting still with the display on, and continuous walking.

Condition	Polling	Interrupt	Saving
Still, display off	[TODO:]	[TODO:]	[TODO:]
Still, display on	[TODO:]	[TODO:]	[TODO:]
Walking	[TODO:]	[TODO:]	[TODO:]

Table 15: Average current with the IMU polled continuously versus woken by INT1. The still-with-display-off row is the one that matters: it is the condition a wearable spends most of its life in, and the only one where the two paths should differ substantially.

Interpretation. [TODO: The expected result is a clear saving in the still rows and near-parity while walking, since the active window samples at the same rate the polling path always used. If the walking rows differ noticeably, the active window is being opened or held longer than the motion warrants and ACTIVE_WINDOW_MS should be re-tuned.]

7.7 Threats to validity

- **The expansion protocol was verified against a mock, not physical module hardware.** The mock is great for testing the host logic thoroughly (including weird faults a real well-behaved module wouldn't produce), but it doesn't simulate real-world electrical issues like contact bounce timing, clock stretching by a slow target, trace capacitance, or hot-plug brownouts. [TODO: If a physical module was built, say so and report it; if not, this is the single largest gap and should be named as the first item of future work.]
- **No Revision B measurements exist yet.** Since Revision A couldn't run firmware, all our functional claims about the custom board itself are still pending.
- **Walking current is not the platform floor.** Even with the wake interrupt working, our step detection still runs on the application core at 10 Hz. Measuring current while walking will overstate the power draw compared to what the hardware would use if we offloaded step counting completely to the sensor's embedded pedometer.
- **Single-unit sample.** All our electrical measurements come from one assembled board, meaning we haven't characterized any component tolerances or assembly variations.

8 Discussion and comparison with prior work

In this section, we compare our project against existing wearable platforms and expansion ecosystems to clearly define what we actually contributed.

8.1 Comparison with existing wearable platforms

Property	PineTime	Thingy:53 / DK	ZSWatch	FinWatch
Wearable form factor	Yes	No	Yes	Yes
Open schematic	Yes	Yes	Yes	Yes
User-replaceable firmware	Yes	Yes	Yes	Yes
Dual-core radio isolation	No	Yes	Yes	Yes
SoC integration	Module	Module	Module	Bare IC
RF front end designed in-house	No	No	No	Yes
Pre-certified radio	Yes	Yes	Yes	No
Integrated PMIC + fuel gauge	Partial	Yes	Yes	Yes
Sensor complement	Basic	Rich	Rich	Basic
Project maturity	Product	Product	Mature	Prototype
Expansion interface	None	Headers	DK headers	Pogo, on-watch
Expansion without opening the case	No	n/a	No	Yes
Third-party modules (no first-party driver)	No	No	No	Yes
Runtime device discovery	No	No	No	Yes
Self-describing descriptor	No	No	No	Yes

Table 16: FinWatch against comparable open wearable platforms. Above the line, FinWatch is conventional or behind - ZSWatch in particular shares the SoC, RTOS, PMIC and flash part, carries more sensors, and is a far more mature project. The claim of this work is confined to the rows below the line, and specifically to the last three: not that the watch can be expanded, but that it can use hardware whose existence its firmware does not know about.

Looking at the comparison above, it is important to be clear about our claims. FinWatch is absolutely not a “better” smartwatch than a mature open-source product like the PineTime. If we compared them on battery life, mechanical finish, and manufacturing maturity, our first-revision student board would lose comprehensively. However, what FinWatch has that those platforms do not is exactly what is highlighted in the bottom three rows of Table 16: a defined interface that lets a sealed device acquire new hardware capabilities it didn’t ship with, alongside a safe protocol that makes that addition possible without prior firmware knowledge.

8.2 ZSWatch in particular

We need to discuss ZSWatch (Section 1.2) separately because it is the closest prior work to our project, and frankly, it is a stronger project on the hardware side. It uses the exact same SoC, RTOS, PMIC, external flash, and originally specified IMU as we do, but it adds a barometer, magnetometer, ambient-light sensor, and microphone, and it has already iterated through multiple hardware revisions. Claiming that FinWatch is a “better open Zephyr smartwatch” would just be false.

Module versus bare silicon. The biggest hardware difference between the two projects actually runs in the opposite direction. ZSWatch integrates the nRF5340 by using a u-blox NORA-B10 module. This is a pre-certified assembly that already contains the SoC, the 32 MHz crystal, the DC/DC converter components, the RF matching network, and — in the NORA-B106 variant — an integrated PCB antenna; the optional 32.768 kHz crystal still has to be placed on the host board. We built FinWatch using the bare nRF5340-QKxx IC, meaning we had to place the crystals, load capacitors, converter inductors, and the entire RF matching network directly on our main board (Figure 3). This choice had massive consequences for our project:

- **RF design became our problem.** A module comes with characterized radio performance, but working from bare silicon meant we had to own the matching network, antenna keepout, and feed-line geometry, any of which can silently ruin radio sensitivity.
- **Certification.** Modules come pre-certified. A bare-silicon design carries the full regulatory burden, which is irrelevant for a student prototype but a dealbreaker for a commercial product.
- **Manufacturing complexity.** A module has coarse pads that easily route on two layers. Escaping the bare aQFN-94 chip is exactly what forced us into a 4-layer board and the expensive via-in-pad process we detailed in Section 5.2.1.
- **Area and cost.** We recovered the cost of this difficulty in board area and unit cost. Our discrete setup takes up less space and is cheaper per unit than a module, which was crucial for fitting inside a 45.97 mm circular board.

Neither approach is inherently better. ZSWatch made the correct engineering choice to guarantee a working radio and focus their effort elsewhere. We deliberately chose the bare IC route to keep the RF front end, power tree, and fine-pitch escape routing within the scope of our project, because we felt it was a more instructive challenge for a final year computer engineering project.

Expansion. The other major difference is how we handle expansion. ZSWatch has an Extension PCB, but it plugs into the breakout headers of their separate WatchDK development board, and it requires first-party drivers written specifically for it. Our FWMP targets a completely different scenario: attaching a module the firmware has never seen before to a sealed watch, and making it usable without any host-side code because the module describes itself. Both are valid answers to different problems.

It is also worth highlighting the convergence here: two independent projects chose the exact same core components without sharing any design work. This suggests that the design space for an open Zephyr wearable is heavily constrained, and that the real novelty lies in what the device is built to do — which is where we spent our design effort.

8.3 Comparison with existing expansion ecosystems

Against Qwiic and STEMMA QT. The difference is discovery. Those ecosystems standardize the physical connector and voltage, but the host still needs a hardcoded driver for every single device. FWMP standardizes the description as well, so an unknown module becomes instantly usable.

Against USB HID. We borrowed the self-describing descriptor idea from USB HID, but HID’s parser and bus stack are way too large and power-hungry for a smartwatch. FWMP implements this concept using a lightweight TLV format that requires less than 100 lines of C code to parse.

Against SMBus/PMBus. Those standards identify a device just so the host knows which existing driver to select. FWMP’s descriptor carries enough operational info that no driver is needed at all.

8.4 What is new

To summarize, here are the specific novel contributions of this project:

1. A wearable-scale expansion contract that works through a sealed enclosure using only four contacts.
2. A self-describing descriptor small enough to be served by an 8-bit microcontroller, featuring forward-compatible TLV extensions.
3. An explicit trust boundary at the connector that treats module data as adversarial input and strictly validates it before use — an approach that is standard in networking but very rare in hobbyist hardware buses.
4. A power-versus-latency polling policy tied directly to user attention, which only honors module polling hints in the direction that saves battery.

8.5 Limitations

Our current implementation has several known limitations:

- It only supports one module at a time at a fixed address. Supporting multiple modules would require dynamic addressing or a hub, which FWMP v1 does not support.
- The port is polled only, which creates a standing idle current cost and an attach latency of up to two poll intervals.
- [I²C](#) bandwidth restricts the port to low-rate sensing. Streaming data like audio or video is out of scope by design.
- Our trust boundary is completely logical. While the protocol protects the firmware, there is no electrical over-current or ESD protection on pin 1 to stop a bad module from short-circuiting the 3.3 V rail.
- Our protocol verification was done against a software mock rather than physical module hardware (as discussed in [Section 7](#)).

9 Future research directions

Here are the next steps and potential improvements for the project:

1. **Build physical modules.** The immediate next step is to actually build a physical module and implement our M1–M8 contract on a small microcontroller. This will let us test the real-world electrical issues that our software mock couldn't simulate, like contact bounce during hot-plugging, clock stretching, supply transients, and the capacitance of longer bus traces.
2. **Move step counting entirely into the IMU.** Right now, the host wakes up and samples at 10 Hz during motion, but we could offload this completely to the LSM6DSOX's embedded pedometer. The step counter could just be read as a register, meaning the CPU wouldn't even need to wake up while walking. We could also use the sensor's Machine Learning Core to classify activity (like walking, running, or standing still) and use the 9 KiB FIFO to batch raw samples for when the host actually needs them.
3. **Complete Revision B bring-up.** Once we get the new board with the restored SWD debug access, we need to repeat Experiment 1 directly on the custom hardware. We also need to measure the real current consumption per rail and validate the RF matching network against our on-board antenna.
4. **Electrical protection at the port.** We need to add current limiting and ESD protection on pin 1 and the signal lines. Since the pogo pins are on the outside of a watch, they will get touched, exposed to sweat, or shorted by keys in a pocket. Our logical trust boundary needs a physical electrical counterpart to be fully safe.
5. **Multiple concurrent modules.** We want to explore a discovery scheme that supports more than one module at a time. This could involve assigning addresses from a pool during enumeration or scanning a documented address range, all without adding more physical contacts to the watch.
6. **Interrupt-capable variant.** We should evaluate either adding a fifth physical contact or overloading one of the existing lines to act as an attention signal in the idle state. This would completely eliminate the idle polling current cost and reduce our attach latency.
7. **Richer descriptor semantics.** We could add new TLV types to carry physical units, value ranges, and sample scaling. This would let the watch actually plot a module's data meaningfully instead of just displaying raw bytes. Because of our forward-compatible design, older watches would safely skip these new TLVs and keep working.
8. **Module authentication.** Adding a challenge-response TLV would let the watch distinguish a certified module from an arbitrary one. This is critical if the platform is ever going to be used for medical or safety-relevant sensing.
9. **Power negotiation.** We could have modules declare their required current draw inside their descriptor. This would let the watch refuse a module the battery cannot support (switching pin 1 off would additionally need a load switch), or automatically lower the screen brightness to accommodate the extra load.

10 Conclusion

This project set out to determine whether a wrist-worn device could be built as an extensible platform rather than a sealed product, and to do so without abandoning the constraints - size, power, integration - that make wearables difficult in the first place.

The hardware answer is a custom four-layer board integrating an nRF5340 SoC with its RF front end and dual crystals, an nPM1300 PMIC providing two switched-mode rails together with Li-Po charging and Coulomb-counter fuel gauging, an LSM6DSOX six-axis IMU, 512 Mbit of QSPI flash, an SPI display with capacitive touch, and two pogo-pin interfaces for charging and expansion.

The software answer is roughly 2550 lines of Zephyr C across nine subsystems, structured so that hardware description lives entirely in the Devicetree and the application is portable between an evaluation kit and the custom board.

The contribution is the FinWatch Module Protocol: four contacts, a fixed I²C address, and a CRC-protected self-describing descriptor through which an unknown module announces its name, capabilities, report geometry, and preferred poll rate - so that a sealed watch can use hardware its firmware has never heard of.

The design treats the connector as a trust boundary, validating every module-supplied byte before use, and confines the port to the lowest-priority thread on a system whose radio runs on a separate core, so that no module behaviour can propagate into the watch's real-time obligations.

The verification is partial and this report says so plainly.

The firmware, including the complete protocol implementation and all six of its rejection paths, was verified on an nRF5340-DK.

The manufactured revision-A board's power architecture was verified electrically, but a missing SWD header made it unprogrammable, forcing a second revision whose bring-up remains outstanding. That defect produced the pre-fabrication checklist of Section 6.7, and the observation behind it - that a rules check validates the connections that exist but cannot demand the ones that should - is the most transferable single result of the project.

References

- [1] Nordic Semiconductor. *nRF5340 Product Specification*. [TODO: version, year]. <https://docs.nordicsemi.com/>
- [2] Nordic Semiconductor. *nPM1300 Product Specification*. [TODO: version, year].
- [3] STMicroelectronics. *LSM6DSOX: iNEMO 6DoF Inertial Measurement Unit with Machine Learning Core and Finite State Machine*, Datasheet. [TODO: revision, year].
- [4] STMicroelectronics. *AN5259: LSM6DSOX Machine Learning Core*, Application Note. [TODO: revision, year].
- [5] Bosch Sensortec. *BMI270 Datasheet: Smart Low-Power Inertial Measurement Unit*. [TODO: version, year]. (Original IMU selection; see Section 6.)
- [6] Macronix. *MX25U51245G 512Mb 1.8V Serial NOR Flash Datasheet*. [TODO: version, year].
- [7] Texas Instruments. *PCA9306 Dual Bidirectional I²C Bus and SMBus Voltage-Level Translator*. [TODO: version, year].
- [8] NXP Semiconductors. *I²C-bus Specification and User Manual*, UM10204. [TODO: revision, year].
- [9] The Zephyr Project. *Zephyr Project Documentation*. <https://docs.zephyrproject.org/>
- [10] Nordic Semiconductor. *nRF Connect SDK Documentation*. <https://docs.nordicsemi.com/bundle/ncs-latest/>
- [11] Bluetooth SIG. *Bluetooth Core Specification*. [TODO: version, year].
- [12] USB Implementers Forum. *Device Class Definition for Human Interface Devices (HID)*, Version 1.11, 2001.
- [13] KiCad Development Team. *KiCad EDA Documentation*. <https://docs.kicad.org/>
- [14] LVGL. *Light and Versatile Graphics Library Documentation*. <https://docs.lvgl.io/>
- [15] Pine64. *PineTime Hardware Documentation*. <https://wiki.pine64.org/wiki/PineTime>
- [16] Espruino. *Bangle.js 2 Technical Reference*. <https://www.espruino.com/Bangle.js2>
- [17] ZSWatch Project. *ZSWatch: Open Source Smartwatch Built on the Zephyr RTOS*. <https://github.com/ZSWatch/ZSWatch> (accessed [TODO: date]).
- [18] IPC. *IPC-2221: Generic Standard on Printed Board Design*. [TODO: revision]
- [19] IPC. *IPC-4761: Design Guide for Protection of Printed Board Via Structures*. [TODO: revision]

A Appendix A - Firmware

The complete firmware source is available at [\[TODO: repository URL\]](#). It is not reproduced here: at roughly 2800 lines across nine subsystems it would add little that Sections 4 and 5 do not already state, and the parts that carry the argument - the FWMP register map, the descriptor format, the validation pipeline, the enumeration state machine and the interrupt-driven wake path - are specified in the body in a form that can be implemented independently of this particular source tree.

Path	Contents
<code>src/expansion/</code>	<code>expansion_proto.h</code> (FWMP wire format, no Zephyr dependencies, intended to be shared verbatim with module-side firmware); <code>expansion.h/.c</code> (enumeration, validation, poll loop); <code>expansion_transport.h/.c</code> (I ² C backend); <code>expansion_mock.c</code> (canned descriptor and fault injection)
<code>src/sensor/</code>	Wake-up interrupt configuration and active-window sampling
<code>src/display/</code>	LVGL setup, watch face, module screen
<code>src/ble/</code> etc.	Conventional Zephyr peripheral drivers
<code>boards/</code>	Devicetree overlays
<code>prj.conf</code>	Kconfig selections

Table 17: Firmware source layout; per-subsystem sizes are given in Table 12.

B Appendix B - Hardware documentation

[TODO: Add, exported from KiCad:]

- Full schematic sheets (PDF export, one page per sheet)
- PCB layer plots: top copper, bottom copper, inner layers, silkscreen, solder mask
- 3D render of the assembled board, top and bottom
- Photograph of the assembled revision-A board
- Complete bill of materials as a table: reference, value, footprint, manufacturer part number, quantity
- Fusion render of the enclosure

C Appendix C - Reference expansion module

[TODO: A minimal compliant module against the M1–M8 checklist of Section 4: schematic (an 8-bit MCU, the four pogo contacts, the sensor), the descriptor bytes it serves annotated field by field, and the module-side firmware skeleton that answers the register map. This is the section a third party would read to build hardware for the platform.]